

dsh-handbook-zh

DeepSeek Harness 从 0 到 1 · 系统化中文教程

安装 · Profile · 插件 · 开发 · 发布

2026 年 8 月 · github.com/863683348/dsh-handbook-zh

目录

- 01 · 快速开始
- 02 · 理解 Profile
- 03 · 插件入门
- 04 · 写第一个插件
- 05 · 工具开发进阶
- 06 · 发布插件
- 07 · 实战案例
- 08 · 常见问题

01 · 快速开始：30 分钟把 DeepSeek Harness 跑起来

一句话总结：DeepSeek Harness（DSH）是一个"用插件组合出来的 Agent 运行时"，你只需要安装一个`dsh`命令，就能启动 Web 界面、跑无头会话、管理插件——本章带你完成第一次完整的上手，建立"命令 → profile → 工具 → workspace"的全局心智。

本章目标

读完本章并照做，你将：

- 安装并验证`dsh`命令；
- 启动一次 Web 会话（`dsh web`）并发出第一个任务；
- 用`headless`模式跑一个一次性的持久化会话；
- 记住 DSH 最常用的入口命令，并能说出 workspace 与 \$DSH_HOME 的含义。

1. 先建立一个整体印象

DeepSeek Harness 的定位可以概括成一句话：****一个以插件为积木的 Agent 运行时****。它不是一个"聊天软件"，也不是一个"代码生成器"，而是一个平台——你（或社区）用插件给它拼出各种能力，模型在平台上调用这些能力干活。官方口号"一切皆插件"说的就是这个：工具是插件、提示词是插件、界面面板是插件、甚至 Agent 的治理规则也是插件。

命令行工具`dsh`是进入这个平台的钥匙。它本身很"薄"，主要做三件事：

dsh 的职责	说明
启动 profile	`dsh --profile <name>` 把某个 profile 的组合配置加载起来
管理插件	`dsh plugin` 在 profile 里增删插件（转发给 pnpm）
提供别名	`dsh web` 就是 `--profile web` 的别名

你不需要先精通插件才能用 DSH——就像不需要会修车才能开车。本章先把车开起来。

2. 准备工作

你需要一台能跑 Node.js 的电脑（Windows / macOS / Linux 均可）。DSH 本体是一个 npm 包，命令行工具叫`dsh`，所以唯一硬性依赖是 Node.js 与 npm。建议使用当前 LTS 版本的 Node.js，具体版本要求以官方文档为准。

```
# PowerShell■■■■ node ■ npm ■■
node --version
npm --version

# bash■■macOS / Linux / WSL■
node --version
npm --version
```

另外请准备好一个你常用的终端：Windows 上用 PowerShell 或 Windows Terminal，macOS/Linux 上用任意 shell。本书所有命令都给出双写法，绝大多数`dsh`命令在两个 shell 下完全一致，只有涉及路径、引号、环境变量时才需要区分——遇到不一致的地方，[08-常见问题](08-常见问题.md)有专门的对照表。

国内网络提示：npm 默认走官方源，安装慢或失败时可以临时切换镜像源（如`npm config set registry https://registry.npmmirror.com`）。这属于通用 npm 知识，具体以你的网络环境为准。

3. 安装 dsh

`dsh` 是 DeepSeek Harness 中用于启动 profile 的命令，随 npm 包 `@deepseek-ai/dsh` 分发。全局安装它：

```
npm install -g @deepseek-ai/dsh
```

```
npm install -g @deepseek-ai/dsh
```

安装完成后验证：

```
dsh --version
```

```
dsh --help
```

```
dsh --version
```

```
dsh --help
```

- `dsh --version` 应打印版本号（本书写作时官方版本为 0.1.0-rc.6，以你安装到的版本为准）；
- `dsh --help` 打印**启动器自己的**帮助——注意它只包含启动器 flag，不包含某个具体应用（如 Web 应用）的 flag。这个区别很关键：`dsh --help` 与 `dsh --profile web --help` 打印的是两份不同的帮助，下一章会专门讲。

Windows 提示：如果 `dsh` 提示“不是内部或外部命令”，多半是 npm 全局 bin 目录不在 PATH 里。执行 `npm prefix -g` 找到全局目录，把它的 bin 子目录加进 PATH 后重开终端。

需要升级时：

```
npm update -g @deepseek-ai/dsh
```

```
npm update -g @deepseek-ai/dsh
```

卸载则 `npm uninstall -g @deepseek-ai/dsh`（一般用不到，先知道有这回事）。

4. 两个关键概念：\$DSH_HOME 与 workspace

在跑第一条命令之前，先建立两个心智模型，它们贯穿全书。

****\$DSH_HOME——DSH 的家。**** profile 目录、全局配置（home 级 `cordis.patch.yml`）、会话数据都存在这里。未设置时默认是用户主目录下的 `.dsh`：

```
# PowerShell■■■■■■■■■■
echo "$env:USERPROFILE\.dsh"
```

```
# bash
echo "$HOME/.dsh"
```

第一次启动后你可以进去逛一逛，会看到 `profiles/` 目录（每个 profile 一个子目录）。profile 结构在 [02-理解-profile](02-理解-profile.md) 详细拆解。

****workspace——Agent 干活的工作区。**** **运行 `dsh` 命令时所在的目录就是默认 workspace 根**。

也就是说，你在哪个目录启动 DSH，Agent

默认就在哪个目录里读文件、写文件、跑命令。想让它处理某个项目，就先 `cd` 进那个项目再启动。

为什么强调 workspace？因为“Agent 只在自己的工作区里动手”是 DSH

的安全模型之一：插件写的文件、跑的命令都默认圈定在 workspace 里。第 05 章讲 `ctx.fs` 的 containment

校验时你会再次遇到它。一个直观的验证方法：在空目录 A 启动 `dsh web` 让模型建文件，再换到空目录 B 启动——模型不会“记得”A 里的文件，因为 workspace 不同、会话也不同。

5. 第一次启动：dsh web

``dsh --profile headless "job"` 会**运行一个全新的持久化会话**（persistent session），完成作业后**打印最终答案并退出**。它和 web 一样首次使用自动初始化。三个值得注意的点：

- 1. **全新**：每次调用都开一个全新会话，不会带上上次对话的上下文（除非通过持久化机制，如插件写的工作区文件）；
- 2. **持久化**：会话本身是持久化的（有会话 ID、可恢复），不是无状态的一次性调用；
- 3. **打印答案退出**：适合脚本化——你可以把它嵌进批处理、CI、定时任务里，当成一个"带 Agent 的 CLI"用。

引号注意：PowerShell 与 bash 都要求把整个任务文本用引号包起来。如果任务文本里含有双引号，PowerShell 建议外层用单引号，bash 则注意转义，详见 [08-常见问题](08-常见问题.md)。

****6.1 web 还是 headless？一张表说清楚****

维度	<code>`dsh web`</code>	<code>`dsh --profile headless "任务"</code>
界面	浏览器图形界面	无界面，纯命令行
会话	可多会话、可追问、上下文连续	每次全新会话，打印答案退出
适合	交互式工作、需要反复确认	一次性任务、脚本/CI/批处理
初始化	首次自动	首次自动
典型姿势	人坐在电脑前	人不在场，或嵌入自动化

简单记：****人在就 web，人不在就 headless****。两者共享同一个 profile 体系（headless 对应 ``headless`` profile），装在同一 profile 里的插件两边都能用——插件知识从 03 章开始。

7. 常用入口命令速查表

命令	作用	什么时候用
<code>`dsh --version` / <code>`dsh --help`</code></code>	版本 / 启动器帮助	环境自检、忘了 flag 时
<code>`dsh web`</code>	启动 Web 版	日常交互式使用
<code>`dsh --profile web --port 8080`</code>	指定端口启动 Web 版	默认端口被占用时
<code>`dsh --profile headless "任务"</code>	跑一次性无头会话	脚本化、批量化、CI
<code>`dsh --profile <name>`</code>	启动任意 profile	用自定义 profile 时
<code>`dsh plugin --profile <name> <pnpm args>`</code>	管理该 profile 的插件	装/卸插件（03 章）
<code>`dsh --dump-config`</code>	打印组合后的配置树	排查配置问题（02 章）

8. 第一个完整任务示例

把前面的概念串起来，完整走一遍"建目录 → 启动 headless → 拿到产物 → 验证"：

```
# PowerShell
mkdir D:\scratch\dsh-demo
cd D:\scratch\dsh-demo
dsh --profile headless "■■■ hello.txt■■■■■ 'Hello from DeepSeek Harness'■■■■■■■■■■"
Get-Content hello.txt
```

```
# bash
mkdir -p ~/scratch/dsh-demo
cd ~/scratch/dsh-demo
dsh --profile headless "■■ hello.txt■■■■ 'Hello from DeepSeek Harness'■■■■■■■■■■"
cat hello.txt
```

****预期过程与结果：****

- 1. headless 会话在 workspace（即`dsh-demo`目录）里创建`hello.txt`；
- 2. 模型读取该文件并打印内容；
- 3. 会话打印最终答案后退出；
- 4. 你手动`cat` / `Get-Content` 验证文件确实存在于当前目录。

这就是 DSH 的最小闭环：****命令 → profile → Agent 工具 → workspace 产物****。注意观察：所有文件操作都发生在`dsh-demo`里，因为启动命令的目录就是 workspace 根。

9. 常见误解（先打个预防针）

误解	事实
"DSH 是另一个 ChatGPT 网页"	它是有文件系统、命令执行、插件体系的 Agent 运行时，Web 只是入口之一
"`dsh web` 要手动配置"	web / headless 首次使用自动初始化，零配置
"`dsh --help` 能看到全部参数"	那只是启动器帮助；应用参数要看`dsh --profile web --help`
"模型在哪都能写文件"	默认只在自己的 workspace 里动手，路径解析受控

10. 新手检查清单

- [] `npm install -g @deepseek-ai/dsh` 成功，`dsh --version` 有输出
- [] 知道`\$DSH\_HOME`指向哪里（默认`~/dsh`），第一次启动后见过`profiles/`目录
- [] `dsh web` 能启动并在浏览器打开界面，能完成"创建 README"任务
- [] `dsh --profile headless "任务"` 能打印答案并退出
- [] 理解"运行命令的目录 = 默认 workspace 根"，并在实验里验证过
- [] 区分`dsh --help`（启动器）与`dsh --profile web --help`（应用）

11. 下一步

你已经能跑 DSH 了，但"它为什么长这样"还不清楚。下一章 [02-理解-profile](02-理解-profile.md) 拆解 profile 的结构、配置层的叠加顺序，以及如何用`--dump-config`检查组合结果——这是理解后面所有插件知识的地基，值得花 20 分钟认真读。

02 · 理解 profile：DSH 的配置是如何组合出来的

一句话总结：profile 是 DSH 的"配置单元"——一个目录里装着插件清单与补丁，启动时按固定顺序把多层 patch 叠加成最终配置树，理解这个叠加顺序（以及用 `--dump-config` 观察它）是玩转 DSH 的关键。

1. profile 是什么

`dsh` 本身几乎不做业务：**profile 由多个插件组合包（bundle）的 patch 层按顺序叠加而成，其上再应用用户自己的覆盖配置**。启动一个 profile，就是把把这个组合结果交给运行时去执行。

可以这样类比：profile 之于 DSH，就像"环境"之于开发工具——它决定了一套会话里有哪些工具、什么提示词、什么行为。只是在这里，构成环境的不是配置文件里的一个开关，而是一层一层**叠出来的 patch**。

一个 profile 对应 `\$DSH\_HOME/profiles/<name>` 目录。启动命令：

```
dsh --profile <name>
dsh --profile <name>
```

内置的 `web` 与 `headless` 两个 profile 首次使用时**自动从随附模板初始化**；**其他任何 profile 都必须通过 `dsh plugin` 创建**（详见第 7 节与 [03-插件入门](03-插件入门.md)）。

2. profile 目录结构

一个典型的 profile 目录长这样：

```
$DSH_HOME/profiles/<name>/
■■■■ package.json          # ■■■■out-of-tree■■■■■■■■■■
■■■■ dsh.profile           # profile manifest■■■■ + ■■■■■■ bundles ■■■
■■■■ cordis.patch.yml      # ■■■■■ patch ■
■■■■ node_modules/        # pnpm ■■■■■■■■■■ dsh plugin ■■■
■■■■ ...                   # ■■■■■■■■■■■■■■■■■■
```

三个关键文件的职责：

文件	作用	由谁维护
`package.json`	声明 profile 依赖的 树外插件 （不在 dsh 安装目录里的插件）	`dsh plugin`（转发 pnpm）自动维护
`dsh.profile`	manifest：元数据 + `bundles` 列表（组合包按顺序排列）	创建 profile 时生成；插件增删时自动 reconcile
`cordis.patch.yml`	用户自己的 patch 层，追加在组合包 patch 之后	你自己手写

逐一展开：

- package.json 的"树外插件"**：指那些不在 dsh 安装目录里的插件包。DSH 安装目录自带官方组合包，第三方插件则作为依赖装进 profile 自己的 `node\_modules`。`dsh plugin` 的 add/remove 本质就是在维护这份依赖清单；
- dsh.profile（manifest）**：记录 profile 的元数据（名字、描述等）和**按顺序排列的 bundles 列表**。`bundles` 列表的顺序就是叠加顺序，插件安装/卸载时该列表会被自动 reconcile（比如 `dsh plugin add` 一个本地目录时会自动把它的组合包加进列表）。manifest 的具体字段格式以官方文档为准；

- ****cordis.patch.yml****：你的覆盖层。组合包提供了"出厂配置"，你想改什么、加什么都写在这里。

3. 组合包从哪解析

`dsh.profile.bundles` 里列出的组合包，解析顺序是：

1. 先从 ****dsh 安装目录****解析内置组合包：`@deepseek-ai/dsh-base`、`@deepseek-ai/dsh-web-app`、`@deepseek-ai/dsh-headless`；
2. 再从 ****profile 自身的 node_modules**** 解析（pnpm 会把树外插件安装到这个目录）。

为什么要分两步？因为内置组合包随 DSH 版本升级、由官方统一维护，不该被某个 profile 的依赖"绑架"；而第三方插件装在 profile 里，每个 profile 可以有自己的一套（甚至可以装不同版本）。****一个 profile 的插件生态 = 内置组合包 + 树外插件****，这个等式帮你快速判断"某个能力从哪来"。

4. 配置层叠加顺序（重点）

配置树以****空根****为起点，依次叠加以下配置层：

```

■■ (empty root)
  ▼ ■ 1 ■■■dsh.profile.bundles ■■■■■ patch■■ bundles ■■■■■
  ▼ ■ 2 ■■■profile ■■■ cordis.patch.yml
  ▼ ■ 3 ■■■$DSH_HOME/cordis.patch.yml■■home ■■■■■
  ▼ ■ 4 ■■■--patch ■■■■■
  ▼ ■■■■■

```

理解要点：

- ****先底层后上层，上层覆盖下层****：同一个配置键在多层都出现时，后叠加的层生效。比如官方组合包给某工具设了默认值，你在 profile 的 `cordis.patch.yml` 里改它，就会覆盖出厂值；
- ****组合包的 patch 由官方包提供****，不要直接改它们——你的覆盖写在第 2~4 层；
- ****`\$DSH\_HOME/cordis.patch.yml` 是对所有 profile 生效的全局层****：公司统一规范、个人通用偏好放这里；
- ****`--patch` 是命令行级临时覆盖****：适合调试，不落盘，重启就没了。

****为什么是"patch 叠加"而不是"一份大配置文件"？**** 这是 DSH

插件体系的根基：如果配置是一整份文件，那么每个插件都要"改别人的文件"，装两个插件就会打架。而 patch 是****增量****——每个组合包只声明自己关心的片段，运行时按顺序把它们拼起来，冲突时后层覆盖前层。这保证了：插件可以独立开发、独立发布、任意组合；用户的覆盖永远有地方放（第 2~4 层）；官方升级组合包时不会覆盖掉你的修改。理解了这一点，你就理解了整个 DSH 配置模型。

****一个具体的叠加例子（强烈建议动手试一遍）：****

假设某内置组合包在 patch 里写了某工具的最大重试次数：

```

# ■ 1 ■■■■■■■ patch■■■■
- tool-retry:
  maxRetries: 3

```

你在 profile 自己的 `cordis.patch.yml`（第 2 层）里覆盖它：

```
# ■ 2 ■■profile ■■ cordis.patch.yml■
- tool-retry:
  maxRetries: 5
```

那么 `dsh --profile <name> --dump-config` 里该键的最终值是 ****5****——因为第 2 层压在第 1 层之上。而如果第 2 层里没有这一项，最终值就是 3。这个"查最终值"的过程，就是 `--dump-config` 最常用的用途：****它告诉你最终生效的到底是什么，省得你一层层去翻 patch****。

5. 启动器 flag 与应用参数

启动器只解析自己的 flag，并把****其后的一切****交给被启动的 profile（由 profile 里的应用插件解析这份共享的不可变参数快照）。因此：

- 启动器 flag 必须写在最前面；
- 启动器无法识别的第一个 token 标志着应用参数的开始。

```
dsh --profile web --port 8080      # --port ■■ web ■■
dsh --profile headless "run the tests"
dsh --profile web --help           # ■■ web ■■■■■■
dsh --help                         # ■■■■■■■■■■

dsh --profile web --port 8080
dsh --profile headless "run the tests"
dsh --profile web --help
dsh --help
```

这条规则很反直觉，却是排查"参数没生效"的第一现场：如果你把 `--port` 写在了 `--profile` 前面（`dsh --port 8080 --profile web`），启动器会把它当自己的 flag 解析掉——参数顺序错了，应用自然收不到。

再往深一层说：启动器拿到应用参数后，会把它包装成一份****共享的不可变参数快照****交给 profile 里的应用插件去解析。这意味着两件事：其一，应用侧的所有插件读到的是****同一份****参数，不会各读各的；其二，这份快照是****只读的****，任何插件都改不了它，避免插件之间互相干扰。所以 `dsh --profile headless "任务" 里那个任务文本`，headless 应用的所有相关插件都能看到同一份；`dsh --profile web --port 8080` 的端口也一样。参数在启动那一刻就"定型"了，之后不会再变。

6. --dump-config 实践：看见组合结果

官方提供两个开关，****不启动****就能检查组合后的配置树：

开关	作用	什么时候用
`--dump-default-config`	打印默认配置（不叠加用户覆盖）	看"出厂"长什么样
`--dump-config`	打印完整组合后的配置树	看你的覆盖有没有生效

用 `--dump-config` 检查某个 profile 的最终配置：

```
dsh --profile web --dump-config
dsh --profile web --dump-config
```

把输出存成文件慢慢看：

```
# PowerShell■■■■■■■■■■ Out-File ■■ UTF-8■■■■■■■■■■
dsh --profile web --dump-config | Out-File -FilePath dump.yml -Encoding utf8

# bash
dsh --profile web --dump-config > dump.yml
```

****怎么读这份输出？**** 它是组合后的配置树（YAML/JSON 风格）。核心看两件事：

1. ****有没有你要的东西****：装了某个插件后，搜它的 `id` 或包名（例如 `dsh-plugin-focus` 就搜 `focus`），确认组合行存在；
2. ****值对不对****：某配置项的值是不是被你的覆盖层改掉了——如果 `--dump-config` 里不是你想要的值，说明你的 patch 层没写对位置（多半是层级顺序问题）。

这是后面验证插件生效的标准方法，03 章会反复用到。

****常见问题：我改了配置，为什么 `--dump-config` 里没变化？**** 按下面顺序排查：

1. 是不是改错了文件？profile 级配置在 `\$DSH\_HOME/profiles/<name>/cordis.patch.yml`，home 级全局配置在 `\$DSH\_HOME/cordis.patch.yml`——别把两者搞混；
2. 是不是层级太低？如果内置组合包在更上层又写死了同一键，你的覆盖会被压掉；
3. 是不是格式不对？patch 是 YAML，缩进错误会导致整层解析失败（`--dump-config` 会报错或静默跳过）；
4. 是不是没重启？配置在启动时组合，运行中的会话不会热加载（除非用了 HMR 类插件，以官方文档为准）。

7. 多 profile 管理

不同场景用不同 profile 是 DSH 的推荐姿势。对比三个常见 profile：

profile	初始化方式	典型用途
`web`	首次使用自动初始化	交互式 Web 界面
`headless`	首次使用自动初始化	脚本化一次性会话
自定义（如 `dev`、`client-x`）	**必须通过 `dsh plugin` 创建**	专属插件组合、隔离环境

自定义 profile 的创建与插件管理走同一条命令（`dsh plugin` 在 profile 目录里转发给 pnpm）：

```
dsh plugin --profile dev add dsh-plugin-focus
```

```
dsh plugin --profile dev add dsh-plugin-focus
```

****多 profile 的典型场景：****

场景	做法
开发/生产分离	开发 profile 装调试插件，生产 profile 只装必需插件
客户隔离	每个客户一套 profile，插件互不影响
实验	临时建 profile 试插件，不满意直接删目录

注意：自定义 profile 不像 web/headless 那样自动初始化，它依赖 `dsh plugin` 来创建并维护（具体初始化行为以官方文档为准）。给每个场景一套独立 profile，插件的增删互不影响，这是多场景下的基本隔离手段。

****动手练习（5 分钟）：**** ① 在 `dsh web` 所在的 profile 里执行 `dsh --profile web --dump-config | Select-String -Pattern "port"`（bash 用 grep），找出 Web 服务的端口配置在哪一层；② 在你的 profile 的 `cordis.patch.yml` 里加一段覆盖该端口的值，重新 dump 确认变成你的值；③ 删掉这段覆盖。做完这三个动作，你对“叠加与覆盖”的体感就建立了。

8. 本章小结与检查清单

本章的核心就三句话：**profile 是配置单元，patch

是增量，顺序决定胜负**。只要记住叠加顺序（bundles → profile patch → home patch →

--patch）、记住"上层覆盖下层"、记住一切以`--dump-config`

的最终值为准，你就已经超过了大多数新手。如果只带走一个习惯，那就是：**改任何配置之后，先

`--dump-config`

确认再重启**——它一次回答"有没有、对不对、是不是我要的值"三个问题。下面的清单帮你自测：

8.1 本章检查清单

- [] 能说清 profile 目录里 `package.json`、`dsh.profile`、`cordis.patch.yml` 各管什么
- [] 能背出配置层叠加顺序（bundles → profile patch → home patch → --patch）
- [] 知道内置组合包从 dsh 安装目录解析，树外插件从 profile 的 node_modules 解析
- [] 会用 `--dump-config` 查看组合结果，并用 `Out-File -Encoding utf8`（PowerShell）保存
- [] 会区分 `dsh --help` 与 `dsh --profile web --help`
- [] 知道 web/headless 自动初始化、自定义 profile 必须走 `dsh plugin`
- [] 理解"上层覆盖下层"，能解释 `--dump-config` 里值不对时从哪一层查起

9. 下一步

配置的"容器"理解了，接下来往容器里装东西：下一章 [03-插件入门](03-插件入门.md)

讲怎么找到插件、安装/卸载插件，以及如何验证它真的生效。

03 · 插件入门：找插件、装插件、验证生效

一句话总结：DSH 的官方口号是"一切皆插件"，插件生态爆发式增长（上线 72 小时即超过 4300 个）；本章教你在三个入口里找到想要的插件，用 `dsh plugin` 装进指定 profile，再用 `--dump-config` 验证它真的生效。

1. 插件生态速览

在 DSH 里，**插件（plugin）就是一个可安装的"组合包"**: 它带着自己的 patch 层（`cordis.patch.yml`）和实现代码（Cordis 插件），安装进 profile 后自动参与第 02 章讲的配置叠加。生态现状：

- **"一切皆插件"是官方口号：工具、提示词、界面面板、生命周期钩子……都是插件；
- 生态增长极快：DeepSeek Harness 上线 72 小时插件生态即超过 **4300+**；
- `awesome-dsh-plugin` 是官方精选列表，`data/plugins` 目录已有 **1500+** 条目（持续增长中）。

一个插件通常给 Agent 带来三样东西之一（或组合）：

插件能力	说明	例子
模型工具（tool）	给 Agent 新增可调用的函数	`focus` 专注板、`recipe` 配方
提示词段落（section）	注入使用指引，改变模型行为	`focus:instructions`
生命周期钩子 / UI	拦截事件、渲染面板	dsh-gov 的门禁与审计、focus 的 Web 面板

这套机制意味着什么？不妨把"装插件"想成给 Agent 加技能：装一个 `dsh-plugin-focus`，Agent 就多了一个"把目标钉在纸上、跨会话不忘"的能力；装 `dsh-recipe`，Agent 就多了一个"按场景批量推荐插件"的能力；装 `dsh-gov`，Agent 的所有工具调用就多了一道"权限检查 + 审计记录"。你不需要改任何核心代码——这正是"一切皆插件"的威力：平台的能力边界，由插件生态决定，而不是由官方版本迭代决定。装一个 `dsh-plugin-focus`，Agent 就多了一个"把目标钉在纸上、跨会话不忘"的能力；装 `dsh-gov`，Agent 的所有工具调用就多了一道"权限检查 + 审计记录"。你不需要改任何核心代码，插件就是一切。

2. 三个找插件的入口

入口 A：GitHub topic

GitHub 上打了 `dsh-plugin` topic 的仓库就是插件（`deepseek-harness`、`cordis` 等是相关标签）。直接访问：

`https://github.com/topics/dsh-plugin`

在页面里可以按"最近更新""星数"排序，也可以用主题页的搜索框叠加关键词（如 `dsh-plugin memory`）。这是**发现新插件**最好的入口——npm 里只有发布了的包，而 GitHub 上有正在开发中的。

入口 B：npm registry

包名通常带 `dsh-plugin` 前缀。在终端里搜：

```
npm search dsh-plugin
npm search dsh-plugin
```

输出会列出包名、描述、作者。想过滤出真正可安装的，可以再精确一点：``npm search dsh-plugin focus``、``npm search dsh-plugin governance``。注意：`**npm` 搜得到不等于维护活跃**，安装前看一眼包的更新时间与 README。

****入口 C：awesome-dsh-plugin 精选列表****

<https://github.com/awesome-dsh-plugin/awesome-dsh-plugin>

列表按分类组织（``tools``、``memory``、``notify``、``workflow``、``dev``、``fun`` 等），每个条目指向一个插件仓库，README 由脚本从 ``data/plugins`` 生成（这个机制在第 06 章发布时还会用到）。****选插件先看这里****——能进精选列表的至少过了 CI 门槛（仓库 ≥ 1 天、 ≥ 10 commits、声明了 ``dsh.bundle``）。

社区参考：作者 863683348 的插件家族覆盖了多种场景，可作为“一个插件应该长什么样”的标本：``dsh-plugin-focus``（专注板）、``dsh-recipe``（场景配方）、``dsh-gov``（治理套件）、``dsh-plugin-audit``、``dsh-feed``、``dsh-insight``、``dsh-trend-radar``、``dsh-plugin-guide`` 等，均可在 GitHub / npm 搜索到。07 章会深入拆解其中三个。

3. 安装插件

插件按 profile 安装——****装进哪个 profile，只对那个 profile 生效****。语法：

```
dsh plugin --profile <name> <pnpm args>
```

即 ``dsh plugin`` 会在 profile 目录里把后面的参数转发给 pnpm。最常见的两个操作是 ``add`` 与 ``remove``。

从 npm 安装（包已发布）：

```
dsh plugin --profile web add dsh-plugin-focus
```

```
dsh plugin --profile web add dsh-plugin-focus
```

从本地目录安装（开发期，等价于 ``pnpm add <路径>`` 并自动 reconcile 组合包列表）：

```
dsh plugin --profile web add D:\dev\dsh-plugin-focus
```

```
dsh plugin --profile dev add ~/dev/dsh-plugin-focus
```

安装完成后****重启 DSH****（退出正在运行的 ``dsh web`` / headless 会话再启动），插件才会被加载。为什么必须重启？因为 profile 的配置树在****启动时****组合，运行中的会话不会自动热加载。

pnpm 参数都能透传，例如指定版本：``dsh plugin --profile web add dsh-plugin-focus@1.0.1``。更复杂的 pnpm 用法以官方文档为准。

****三种安装来源，什么时候用哪个？****

来源	命令	适用场景
npm 已发布包	<code>`dsh plugin --profile <name> add <包名>`</code>	日常使用、生产环境
本地目录	<code>`dsh plugin --profile <name> add <路径>`</code>	开发调试、未发布插件
指定版本	<code>`dsh plugin --profile <name> add <包名>@<版本>`</code>	锁定版本、回滚

开发期建议一律用本地目录安装：改代码 → 重启 → 验证，循环极快；等验证稳定了再走第 06 章的发布流程。

****装之前确认两件事：****

1. profile 存在吗？web / headless 首次使用自动初始化；自定义 profile 首次用 ``dsh plugin`` 创建；
2. 包名对吗？npm 包名与 GitHub 仓库名不一定相同（如仓库 ``dsh-plugin-focus``，npm 包名也是 ``dsh-plugin-focus``，但有的插件可能带 scope 或别名）——以包 README 里的安装命令为准。

4. 移除插件

```
dsh plugin --profile web remove dsh-plugin-focus
```

```
dsh plugin --profile web remove dsh-plugin-focus
```

移除后同样需要重启。插件卸载会清理它注册的工具、提示词段落与生命周期钩子（Cordis scope 管理，详见第 05 章）——**不需要你手动去删配置**，`dsh plugin remove` 会自动 reconcile 组合包列表。

想观察"安装到底动了什么"? 装完去 `$DSH_HOME/profiles/<name>/` 里看一眼: `'package.json'` 里多了该插件的依赖项, `'dsh.profile'` 的 `bundles` 列表里多了该组合包, `'node_modules/` 里出现了它的安装产物。这三个变化, 正是第 02 章"树外插件"的落地过程——**装一个插件 = 依赖声明 + 组合列表 + 代码落地**, 三者缺一, 插件就不会生效。

5. 验证插件真的生效了

装完别急着信，用两步验证。

****第一步：看配置树。 **`--dump-config`**

输出的是组合后的最终配置树，插件若生效，它的组合行 (id、包名、config) 一定在里面：

```
dsh --profile web --dump-config | Select-String -Pattern "focus"
```

```
dsh --profile web --dump-config | grep focus
```

有输出 = patch 层生效；没输出 = 插件没进组合树，按第 02

章"改了配置为什么没生效"的顺序排查。输出里你会看到类似`- id: focus`、`name: 'dsh-plugin-focus'`

以及它的`config`块——**这三样齐全，才说明 patch 真的被组合进去了**。如果只有 id 没有 config，多半是 patch 格式问题（比如 YAML 缩进错误），回去检查`cordis.patch.yml`。

****第二步：在会话里让模型调用它。***配置树生效只代表"组合完成"，还要验证"运行时可用"。启动`dsh web`（或`headless`），直接让模型用新工具：

用 focus 工具把当前任务的目标记录下来。

如果模型能调用 `'focus set'` 并返回结果，插件就在运行时生效了。在 Web 界面的工具列表里通常也能看到新注册的工具——模型“看不见的插件”等于没装。

5.1 一个完整的"安装 → 验证"实操演示

把上面的步骤串成一段连贯操作，你照抄即可（假设目标 profile 是 `web`）：

```
# █ 1 █
dsh plugin --profile web add dsh-plugin-focus
# █ 2 ████████████████████ focus █
dsh --profile web --dump-config | Select-String -Pattern "focus"
# █ 3 ████████████
dsh web

# █ 1 █
dsh plugin --profile web add dsh-plugin-focus
# █ 2 ████████████████████ focus █
```

```
dsh --profile web --dump-config | grep focus
# ■ 3 ■■■■■■■■■■■■
dsh web
```

第3步启动后在界面里发："用 focus 工具记下：当前目标是学习 DSH 插件开发。"然后观察：模型应调用 `focus set` 并返回包含路径与条目数的结果。这时再到工作区看看，会发现 `.dsh/focus.md` 文件——**配置生效（组合树）与运行生效（工具调用）两个证据都齐了**。

6. 插件管理命令速查表

命令	作用	备注
`dsh plugin --profile <name> add <pkg>`	从 npm 安装插件到指定 profile	装完重启
`dsh plugin --profile <name> add <路径>`	从本地目录安装（开发期）	自动 reconcile bundles
`dsh plugin --profile <name> remove <pkg>`	卸载插件	卸完重启
`dsh plugin --profile <name> <其他 pnpm 参数>`	其他 pnpm 操作（升级、查看等）	以官方文档为准
`dsh --profile <name> --dump-config`	验证组合树里有没有该插件	排查第一现场

注意事项：

- profile 必须已存在（web/headless 自动初始化；自定义 profile 首次通过 `dsh plugin` 创建）；
- 装完/卸完都要重启 DSH 才生效；
- 内置组合包（`@deepseek-ai/dsh-base` 等）**不要**用 `dsh plugin` 去动，它们由 dsh 安装目录解析、随版本升级。

7. 装哪些？先看精选清单

不知道自己需要什么插件？三条参考路径：

1. **看 awesome-dsh-plugin 的分类**：按你的场景找分类（记忆、通知、工作流……）；
2. **用 `dsh-recipe` 的场景配方**：装 `dsh-recipe` 后直接让模型"给我配一套学术写作环境"，它会给出有序插件清单和安装命令（07 章详述）；
3. **看本仓库的精选推荐**：配套的 `starter/recommended-plugins.md` 按场景整理了经过验证的插件清单，适合新手起步。

动手练习（10 分钟）：**①** 用 `dsh plugin --profile web add dsh-recipe` 装上场景配方插件；**②** `--dump-config` 确认组合行；**③** 在会话里让模型 `recipe list`，看看内置的 10 个配方；**④** 让模型 `recipe apply` 一个你感兴趣的配方，观察它返回的有序插件清单和安装命令；**⑤** 最后 `dsh plugin --profile web remove dsh-recipe` 卸掉。这一圈走完，装、查、用、卸四个动作你都亲手做过了。

8. 常见问题速答

问：装完插件重启后模型还是不调用它？******
先确认描述：工具的描述（description）是否说清了"何时用"？如果描述含糊（比如只说"提供 focus 功能"），模型不知道在什么场景调用。其次确认提示词段落有没有注入——第 05 章会讲怎么用 `ctx.systemPrompt.section` 教模型用工具。

****问：**`dsh plugin add` 报 profile 不存在？****** web / headless 首次使用才自动初始化；自定义 profile 第一次用 `dsh plugin` 创建。如果是自定义 profile，先执行一次 `dsh plugin --profile <name> add <任意包>` 让它初始化，再装目标插件（具体行为以官方文档为准）。

****问：**同一插件能装到多个 profile 吗？****** 能。插件按 profile 安装，互不干扰；同一插件在不同 profile 里可以有不同的 `config`（因为每个 profile 的 `cordis.patch.yml` 是独立的）。

****问：**`npm search` 找不到某个插件？****** 可能它还没发布（只在 GitHub 上）、名字不同（换过名或带 scope），或刚发布索引未更新。去它的 GitHub 仓库 README 看确切的安装命令。

****问：**装了很多插件会不会拖慢启动？******

会有一点。每个插件都要参与配置组合与运行时加载，装得越多启动越慢、潜在冲突越多。建议：按 profile 划分职责（开发/生产分开），用不上的插件及时 remove，保持每个 profile "小而精"。这也正是 `dsh-recipe` 这类 "场景配方" 插件存在的意义——按需组装，而不是全装。

9. 本章检查清单

- [] 能在三个入口（GitHub topic / npm / awesome 列表）找到插件
- [] 会用 `dsh plugin --profile <name> add <pkg>` 安装并重启
- [] 会用 `dsh plugin --profile <name> remove <pkg>` 卸载并重启
- [] 会用 `--dump-config` 验证插件进入组合树
- [] 会在会话里让模型调用新工具做运行时验证
- [] 知道内置组合包不要手动动
- [] 知道装前确认 "profile 存在、包名正确"

10. 下一步

只会装不会写，等于只会用别人的螺丝刀。下一章 [04-写第一个插件](04-写第一个插件.md) 开始动手：三件套骨架逐文件拆解，写一个最小可运行的 `hello` 插件。

04 · 写第一个插件：三件套骨架与 hello 工具

一句话总结：一个可安装的 DSH 插件只有三个文件——`package.json`（声明"我是组合包"）、`cordis.patch.yml`（把自己插入组合树）、`lib/index.js`（Cordis 插件本体），本章逐个文件拆解并写一个最小可运行的`hello` 插件，装进本地 profile 验证。

1. 先建立正确的心智

写插件前先想清楚一个模型：****插件 = 组合包 + 代码****。

- ****组合包（bundle）****：负责"进配置树"。`package.json` 里的 `dsh.bundle.patch` 声明"我的补丁是哪个文件"，`cordis.patch.yml` 里的 `insert` 把自己追加进组合列表——这一步决定了插件会不会被加载；
- ****代码****：负责"提供能力"。`lib/index.js` 是一个标准的 Cordis 插件，在 `apply(ctx, config)` 里注册工具、注入提示词、挂生命周期钩子。

第 02 章讲过配置叠加：组合包的 patch 是第 1 层。所以你的插件装进 profile 后，它的 patch 会参与叠加，你的工具就出现在 Agent 的工具列表里了。本章用最小例子走通这条链路。

三件套总览：

文件	内容	一句话职责
`package.json`	`dsh.bundle.patch` 指向 `cordis.patch.yml` + `peerDependencies` + `publishConfig.access: public`	声明"这是一个 DSH 插件组合包"
`cordis.patch.yml`	`insert` 组合补丁	把自己（一行组合行）插进组合树
`lib/index.js`	Cordis 插件：导出 `{ name, inject, Config, apply }`	实际注册工具、注入提示词、管理生命周期

目录结构：

```
hello-plugin/  
■■■ package.json  
■■■ cordis.patch.yml  
■■■ lib/  
    ■■■ index.js
```

下面把`hello-plugin` 写出来：它注册一个`hello` 工具，向模型提供"打招呼 + 汇报当前工作区"的能力。

2. package.json：插件的身份声明

```
{  
  "name": "hello-plugin",  
  "version": "0.1.0",  
  "description": "■■■ DSH ■■■■■■■■ hello ■■■",  
  "type": "module",  
  "main": "lib/index.js",  
  "files": ["lib", "cordis.patch.yml", "README.md"],  
  "license": "MIT",  
  "publishConfig": { "access": "public" },  
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } },  
  "peerDependencies": {  
    "@deepseek-ai/cordis": "^4.0.1",  
    "@deepseek-ai/dsh-fs": "^0.1.0-rc.6",  
    "@deepseek-ai/dsh-system-prompt": "^0.1.0-rc.6",
```


- `config`：传给插件 `Config` 的配置对象（可省略，用默认值）。字段名与 `Config` 里定义的对齐，缺省字段自动取默认值。

动手验证：改改 `greeting` 的值，重启后 `hello`

工具的问候语会跟着变——这就是“配置驱动行为”的最小演示。

4. lib/index.js：Cordis 插件本体

```
// hello-plugin/lib/index.js
import z from "@deepseek-ai/schemastery";
import { defineTool } from "@deepseek-ai/dsh-tools";

/** Cordis ██████████ id █████ */
const name = "hello";

/** ████████████████████████ apply */
const inject = ["tools", "systemPrompt"];

/** █████ config █████/████ schema */
const Config = z.object({
  greeting: z.string().default("██████████ DSH █████"),
});

function apply(ctx, config) {
  // 1) ██████████
  ctx.tools.register(defineTool({
    name: "hello",
    description: "████████████████████████████████████████",
    parameters: {
      name: { type: "string", required: true, description: "████████████████" },
    },
    output: {
      schema: {
        type: "object",
        additionalProperties: false,
        required: true,
        properties: {
          message: { type: "string", required: true },
        },
      },
    },
    render: (_args, value) => [{ type: "text", text: value.message }],
  }),
  {
    execute: async (args, exec) => {
      const cwd = exec.agent?.session?.header?.cwd; // ██████████
      return { message: config.greeting + " " + args.name + "██████████" + (cwd ?? "██") + "██" };
    },
    presentCall: (args) => ({ card: "generic", title: "Hello " + args.name, kind: "other", rawInput: args }));
  });

  // 2) ████████████████████████ hello
  ctx.effect(() => ctx.systemPrompt.section({
    name: "hello:instructions",
    order: 5,
    text: "██████████████████████████████████ hello █████",
  })), "hello.section()");
}

export { Config, apply, inject, name };

**导出结构拆解（四个导出缺一不可）：**
```

导出	作用	关键点
<code>`name`</code>	插件名字符串，加载器用它注册	须与组合行 <code>`id`</code> 一致
<code>`inject`</code>	依赖的宿主服务数组	声明了才能用： <code>`tools`</code> 、 <code>`systemPrompt`</code> 、 <code>`fs`</code> 、 <code>`agents`</code> 等
<code>`Config`</code>	用 <code>schemastery</code> (<code>`z.object`</code>) 描述配置 schema	缺省值在这里给， <code>`config`</code> 传进来时已校验
<code>`apply(ctx, config)`</code>	插件主体	注册工具、注入提示词、挂钩子都在这

****apply 内部逐行看：****

再看 ``Config`` 的写法：``z.object({...})`` 来自官方 `@deepseek-ai/schemastery`。它做的事是"声明配置的形状 + 给默认值"：``z.string().default("...")`` 表示该字段是字符串、缺省时用默认值。宿主会拿着组合行 ``config`` 来校验并补全，``apply`` 收到的 ``config`` 一定是"形状正确、缺省已补"的——所以你的代码里可以放心地直接用 ``config.greeting``，不用写任何防御性判断。这与 05 章的工具参数 JSON Schema 是两套体系（配置用 `schemastery`，工具参数用 JSON Schema），别搞混。

****为什么 ``Config`` 校验值得认真写？**** 因为组合行的 ``config`` 来自 ``cordis.patch.yml``，而那个文件是**用户手写的 YAML**——用户可能拼错字段名、给错类型、甚至塞进意外内容。``z.object`` 会在 `apply` 之前把这一切挡住：未知字段被忽略或报错、类型不对直接拒绝、缺省字段补默认值。也就是说，**你在 ``apply`` 里永远拿到一份"干净"的配置**，可以放心用。反过来，如果跳过 ``Config`` 直接读 ``config.greeting``，一旦用户没配，你的代码就崩在运行时——而 ``Config`` 会把错误提前到加载期，报错信息也更友好。

1. `ctx.tools.register(defineTool({...}))`` 注册模型工具。``defineTool`` 是官方 `@deepseek-ai/dsh-tools`` 提供的工厂函数，字段含义：

- ``name``：工具名（模型调用名），**避免与内置工具冲突**（``read``/``write``/``edit``/``glob``/``grep``/``pwsh``/``todo_write`` 等）；
- ``description``：给模型的说明，写清"何时用、怎么用"——描述越具体，模型越会正确调用；
- ``parameters``：参数 schema（JSON Schema 风格，下一章细讲）；
- ``output.schema`` + ``output.render``：返回值规范 + 展示渲染；
- ``execute(args, exec)``：工具实现，返回规范值；
- ``presentCall``：调用记录在 UI 上的展示卡片。

2. `ctx.effect(() => ctx.systemPrompt.section({...}), "hello.section())`` 注入提示词段落。``ctx.effect`` 是生命周期原语：插件卸载时，这段注册自动清理。第二个参数是标签，便于排查。

5. 安装到本地 profile 并验证

5.1 语法自检（发布前随时可做）

```
node --check lib\index.js
node --check lib/index.js
```

沙箱/CI 环境里 `node --test` 可能因为子进程被拦截而报 EPERM，此时改用"主模块方式"直接跑测试文件，详见 [08-常见问题](08-常见问题.md)。

****给 hello 插件补一个最小测试（养成好习惯）：****

虽然本章的插件很简单，但从一开始就把"纯逻辑可单测"的纪律立起来：把问候语拼接这类纯函数抽到 `lib/greet.js`，然后写 `test/greet.test.mjs` 用 `node:test` 直接测。07

章拆解的三个真实插件全部遵循这个模式——`lib/\*.js` 零 DSH/Cordis 依赖、可独立单测，`lib/index.js` 只做接线。这一步虽然现在看起来“小题大做”，但它会在你写第二个、第三个插件时省下大量调试时间——纯函数测起来快、定位准，是性价比最高的测试投资。

5.2 装进 profile

用本地路径安装（开发期标准姿势，等价于 `pnpm add <路径>` + 自动 reconcile bundles 列表）：

```
# PowerShell■Windows ■■■■■■■■
dsh plugin --profile dev add D:\dev\hello-plugin

# bash
dsh plugin --profile dev add ~/dev/hello-plugin
```

5.3 验证三步走

****① 组合树里有没有它：****

```
dsh --profile dev --dump-config | Select-String -Pattern "hello"
dsh --profile dev --dump-config | grep hello
```

****② 重启后跑一次会话，让模型调用工具：****

```
dsh --profile dev "■■ hello ■■■■'■■■'■■■■"
dsh --profile dev "■■ hello ■■■■'■■■'■■■■"
```

****③ 看结果：**** 模型应返回形如 `你好，我是第一个 DSH 插件！新同学（工作区：...）` 的消息，且工作区路径与运行命令的目录一致——工具真的跑起来了。

****预期中间过程（帮你判断故障在哪一步）：**** 第 ② 步执行后，模型应当先"看到"工具列表里有 `hello`（来自组合树 + 提示词段落），然后决定调用它，传入 `name` 参数，`execute` 执行并返回规范值，`render` 把规范值渲染成界面文本，模型把结果组织进回答。如果模型直接说"我没有 hello 工具"，回到第 ① 步查组合树与 `id/name` 一致性；如果模型调用了但结果怪怪的，看 `execute` 返回的对象是否匹配 `output.schema`。

5.4 调配置试试

改 `cordis.patch.yml` 里的 `greeting`，重启后再问一次，问候语会跟着变。这验证了 `config` → `Config` → 工具行为的完整链路。你也可以顺便验证 `inject` 声明的作用：把 `inject` 里的 `systemPrompt` 删掉再启动，`ctx.systemPrompt` 会是 undefined，插件会报错——宿主就是这么保证"依赖就绪才 apply"的。

6. 常见新手错误（提前避雷）

****避雷的正确姿势：****

写完三件套先别急着装，把下表从头扫一遍——绝大多数错误发生在"文件之间的契约"上（id 对不上 name、漏了 bundle 声明、peer 放错位置），而不是代码本身。扫完再装，一次成功率会高很多。

错误	现象	修法
----	----	----

<code>`id` 与 <code>`name`</code> 不一致</code>	插件不加载或加载报错	让 <code>`cordis.patch.yml`</code> 的 <code>`id`</code> = <code>`lib/index.js`</code> 的 <code>`name`</code>
漏写 <code>`dsh.bundle.patch`</code>	<code>`dsh plugin add`</code> 后不生效, <code>dump</code> 里没有	补上 <code>"dsh": { "bundle": { "patch": "./cordis.patch.yml" } }</code>
官方包放进 <code>dependencies</code>	装了两份运行时, 行为诡异	一律放 <code>peerDependencies</code>
忘了 <code>`type: "module"`</code>	<code>import/export</code> 语法报错	补 <code>"type": "module"</code>
装完不重启	模型调不到新工具	重启 DSH
<code>`execute`</code> 返回自由文本	输出不符合 <code>output.schema</code> , 展示异常	返回规范值对象
<code>`inject`</code> 漏声明服务却使用	<code>`ctx.xxx`</code> 是 <code>undefined</code> , 运行时报错	用哪个服务就声明哪个 (<code>tools/systemPrompt/fs/agents...</code>)

这些错误里, **``id`/`name`` 不一致**和**``dsh.bundle.patch``**是最隐蔽的两种: 前者插件加载器会注册失败, 后者 ``dsh plugin add`` 根本不知道这是个组合包。写完后把第 7 节的检查清单从头过一遍, 能挡掉 80% 的新手坑。

7. 新手插件检查清单

- [] ``package.json`` 含 ``dsh.bundle.patch`` 与 ``publishConfig.access: public``
- [] 官方 `@deepseek-ai/*` 全部在 `peerDependencies`
- [] ``cordis.patch.yml`` 用 ``insert`` 追加组合行, ``id`` 与插件 ``name`` 一致
- [] ``lib/index.js`` 导出 `{ name, inject, Config, apply }`
- [] ``node --check`` 通过; ``dsh plugin add <路径>`` 成功
- [] ``--dump-config`` 能看到组合行; 会话里模型能调用 ``hello``
- [] 验证过 ``config`` 改动会反映到工具行为

8. 下一步

``hello`` 只是"能跑", 要写生产级插件还需要掌握: ``defineTool`` 的完整字段、``parameters`/`output.schema`` 的 JSON Schema 细节、``ctx.fs`` 文件读写与工作区 containment、``ctx.effect`` 生命周期、多工具插件。下一章 [05-工具开发进阶](05-工具开发进阶.md) 全部覆盖。

05 · 工具开发进阶：schema、文件系统、提示词与生命周期

一句话总结：生产级插件 = 严谨的 JSON Schema（参数与输出）+ 安全的 `ctx.fs` 文件访问（含工作区 containment 校验）+ 恰到好处的提示词段落 + 可清理的生命周期管理；本章用真实插件 `dsh-plugin-focus` 的写法讲透这四件事，并给出多工具插件的组织方式。

1. defineTool 全字段

从 [04-写第一个插件](04-写第一个插件.md) 我们知道工具通过 `ctx.tools.register(defineTool({...}))` 注册。`defineTool` 的核心字段：

字段	类型	作用
`name`	string	工具名，模型调用时用；**避免与内置工具冲突**（`read`/`write`/`edit`/`glob`/`grep`/`pwsh`/`todo_write` 等）
`description`	string	给模型看的说明：何时用、怎么用、有什么副作用
`parameters`	object	参数 schema（JSON Schema 风格的扁平字段描述）
`output.schema`	JSON Schema	返回值的规范 schema（`type: "object" + `properties`）
`output.render`	fn	把规范值渲染成界面文本：`(_args, value) => [{ type: "text", text }]`
`execute(args, exec)`	async fn	工具实现，返回**规范值**（必须符合 `output.schema`）
`presentCall`	fn	调用记录在 UI 上的展示卡片

几个容易忽略的设计原则：

- description 是"模型的说明书"**：写清楚"什么场景调用、参数怎么填、有没有副作用"。模型每次调用前都会读它，写得好，模型就调得准；写"提供 focus 功能"这种废话，模型基本不会主动用；
- name 是"公共 API"**：一旦发布，改名会让旧会话/其他插件失效。命名用短横线或下划线小写，避免与内置工具撞名；
- presentCall 决定"人看到的调用记录"**：`card`/`title`/`kind`/`rawInput` 四个字段控制 UI 展示，`rawInput` 是原始参数。

推荐的开发顺序是 schema 先行：先写 `parameters` 和 `output.schema`（这就是接口契约），再写 `execute` 的实现，最后补 `render` 和 `presentCall`。顺序反过来的后果是：实现写完了才发现返回值没法表达、参数漏了字段，被迫返工。三个官方插件（focus/recipe/gov）都是先有清晰的 schema 再有实现——你在 07 章会看到它们的 schema 如何精确到每个字段的 `required`。

2. parameters：扁平字段式 JSON Schema

DSH 模型的工具参数用 JSON Schema 描述。官方插件（如 focus、recipe、gov）的写法是**扁平字段式**：每个字段一个键，值是该字段的约束：

```
parameters: {
  action: {
    type: "string",
    required: true,
    enum: ["set", "get", "append", "clear"],
```

```

    description: "■■■■■■■■■■",
  },
  note: {
    type: "string",
    description: "■■■■■set / append ■■■■get / clear ■■■■",
  },
}

```

常用约束键：

键	说明	例子
`type`	字段类型	"string"、"integer"、"boolean"、"array"
`required`	是否必填	`true`（必填）
`enum`	枚举取值	`["set", "get", "append", "clear"]`
`description`	给模型的字段说明	写清楚何时必填、何时忽略
`items`	数组元素类型	`items: { type: "string" }`

****为什么是扁平字段式？****

因为对模型来说，扁平的、一眼能数清的参数表比嵌套的复杂结构更容易正确填充。DSH 工具设计的通用建议：

1. ****动作收敛为`enum`****：focus 的`action`、recipe 的`action`、gov 的`action` 全都是枚举——枚举直接告诉模型"只能选这几个"，错误率断崖式下降；
2. ****字段越少越好****：能省则省，可选的字段写明"何时忽略"；
3. ****required 只标真正必填的****：标多了模型会为了凑参数而胡填。

3. output.schema + render：让返回值"规范且好看"

工具返回的必须是****规范值****（canonical value）——即符合`output.schema` 的对象，而不是自由文本。规范值的好处：后续环节（审计、UI、其他工具）可以结构化地消费它。

```

output: {
  schema: {
    type: "object",
    additionalProperties: false,
    required: true,
    properties: {
      action: { type: "string", required: true },
      path: { type: "string", required: true },
      entries: { type: "integer", required: true },
      changed: { type: "boolean", required: true },
      board: { type: "string" },
    },
  },
  render: (_args, value) => {
    const head = "Focus " + value.action + ": " + value.entries + " ■, " + (value.changed ? "■■■■" : "■■■■");
    const lines = [head];
    if (value.board !== undefined) lines.push("", value.board);
    return [{ type: "text", text: lines.join("
") }];
  },
}

```

三个要点：

- **additionalProperties: false**：返回值里不允许 schema 之外的字段，强制规范。漏字段会校验失败，多字段也会——**schema 就是契约**；
- **顶层 required: true**：对象本身必填（不是每个属性都必填）；属性级 **required: true** 才是该属性必填；
- **render 与 schema 分离**：schema 管"数据是什么"，render 管"给人看什么"。render 返回消息块数组（如 `[{ type: "text", text }]`），可以拼多行、加空行、做摘要——注意 render 只是展示，**不要用它承载只有模型才需要的结构化信息**。

4. execute 与 exec 上下文

`execute(args, exec)` 里能拿到什么？看官方插件 focus 的实际用法：

```
execute: async (args, exec) => {
  const cwd = exec.agent?.session?.header?.cwd; // 
  const signal = exec.signal; // 
  // ...  cwd 
  return { action, path, entries, changed, archived };
}
```

要点：

- **exec.agent.session.header.cwd** 是会话工作区——"文件放哪"都以它为基准，绝不要写死绝对路径；
- **exec.signal**：调用被取消时置为 aborted，长任务（大文件、循环、网络）要响应它，否则取消后还在干活；
- 抛 `Error` 即工具失败，模型会看到错误信息——**错误消息要写人话**（"focus: note 是必填的"比 "undefined is not a function" 有用一百倍）；
- 返回对象必须匹配 `output.schema`，这是运行时校验的硬约束。

5. ctx.fs：文件读写与工作区 containment

插件读写文件必须走宿主 ctx.fs 服务，不要直接 `node:fs`。常用方法（以 focus 源码为证）：

方法	作用
<code>ctx.fs.resolve(path, { cwd, signal })</code>	相对 <code>cwd</code> 解析出目标
<code>ctx.fs.contains(target, other)</code>	判断 <code>other</code> 是否在 <code>target</code> 之内（containment 校验）
<code>ctx.fs.stat(target, signal)</code>	取文件信息；不存在返回 <code>undefined</code>
<code>ctx.fs.readText(target, signal)</code>	读文本
<code>ctx.fs.writeText(target, content, undefined, signal)</code>	写文本
<code>ctx.fs.processPath(target)</code>	把目标转成展示用路径

工作区 containment 是安全底线：插件配置里出现的路径（如专注板文件）必须是"相对会话工作区的路径"，并且**永远不能逃逸工作区**。官方插件的标准写法是"解析后校验"：

```
async function resolveTarget(ctx, file, cwd, signal) {
  // 1)  cwd  → 
  const target = await ctx.fs.resolve(file, cwd !== undefined ? { cwd, signal } : { signal });
  if (cwd !== undefined) {
    // 2) 
    const cwdTarget = await ctx.fs.resolve(".", { cwd, signal });
    if (!ctx.fs.contains(cwdTarget, target)) {
```

}

`ctx.fs.contains` 这道闸。dsh-plugin-focus 里板子文件配置成工作区外路径会"fail loud"（直接报错），这是刻意设计——**宁可报错，不可悄悄越界**。

****"写文件先 stat 再 readText 再 writeText"的三步模式****（focus 的 loadBoard/appendArchive 都是这个套路）：`stat` 判断存在性 → `readText` 读旧内容 → `writeText` 写新内容。文件不存在时 `stat` 返回 `undefined`，据此决定是"空内容"还是"读旧追加"。

focus 的 `appendArchive` 就是一个完整示范：`stat` 目标归档文件 → 不存在则以空串开始 → 存在则 `readText` 读旧内容 → 拼接新块 → `writeText` 写回。注意 `append` 类操作**必须读-改-写三步**，而不是只 `writeText` —— 否则会把旧内容覆盖掉。

6. ctx.systemPrompt.section：注入提示词

教模型"什么时候用什么工具、别干什么", 靠提示词段落。注册方式:

```
ctx.effect(() => ctx.systemPrompt.section({
  name: "focus:instructions", // ■■■
  order: 5, // ■■■persona ■ 0■■■■■■■
  text: "The focus tool maintains a durable focus board...",
}), "focus.section()");
```

- ``name``：段落唯一名，惯例是 `<插件名>:instructions` 之类的命名空间式；
- ``order`` 控制段落先后（如 persona 是 0，插件段落常用 5，多个插件按数字排队）；
- 段落文本面向模型，写"何时用、怎么用、注意事项"——focus 的段落就详细教了"任务开始 set、工作中 append、上下文压缩后 get、任务结束 clear"的完整用法；
- 用 ``ctx.effect`` 包裹：插件卸载时段落自动移除，不留垃圾。

****段落与 description 的分工****：`description`

是"这个工具是什么", 段落是"整套工具的用法规范与触发时机"。工具少时段落可以简单 (04 章 hello 只有一句话), 工具多、行为复杂时段落才是主角 (focus、gov 都是)。

7. ctx.effect 与事件钩子：生命周期管理

`ctx.effect(fn, label)` 是 Cordis 的生命周期原语：`fn`

里注册的一切（监听器、段落、定时器）在插件作用域结束时自动清理。注册事件钩子的标准姿势：

```
// ██████████dsh-gov ██████████
ctx.on("tools/pre-execute", async (exec, next) => {
  // ███ { kind: "deny" | "ask", reason } ██████████ next()
  return next();
});
```

```
// ■■■■■■■■■■dsh-gov ■■■■  
ctx.on("tools/result", (exec, result) => {  
    // result.isError ■■■■  
});
```

```
// ■■ Agent ■■■■dsh-gov ■■■■■■dsh-plugin-focus ■■■■■■  
ctx.on("agent/pre-step", async ({ agent, turn, step, signal }, next) => {  
    const decision = await next();  
    if (decision.kind === "reject") return decision;  
    // ■■■■■■■■■■■ plugin snapshot ■■  
    return decision;  
});
```

形态	签名	用途
拦截型	`(payload, next)` 返回 next() 或拒绝决策	`tools/pre-execute` 门禁、`agent/pre-step` 注入
观察型	`(payload, result)` 无返回值	`tools/result` 审计
纯注册	普通 `ctx.on`	监听事件做记录

`inject` 数组声明依赖（如 `["tools", "fs", "systemPrompt", "agents", "tokenMeter"]`），宿主保证这些服务就绪后才 `apply`。以官方包为参考：`dsh-gov` 就 `inject` 了 `tokenMeter` 来做 token

8. 多工具插件：一个插件注册多个工具

```
function apply(ctx, config) {
  ctx.tools.register(defineTool({ name: "tool_a", /* ... */ }));
  ctx.tools.register(defineTool({ name: "tool_b", /* ... */ }));

  ctx.effect(() => ctx.systemPrompt.section({ /* section 1 */ }, "plugin.section1()");
  ctx.effect(() => ctx.systemPrompt.section({ /* section 2 */ }, "plugin.section2()");
  // ████████ apply ██████Cordis scope ██████
}
```

1. ****schema 常量化****：把每个工具的 ``parameters``、``output.schema`` 提成模块级常量，``apply`` 只做装配，可读性大幅提升；
2. ****共享辅助函数****：多个工具都要用"解析 + containment 校验"就把 ``resolveTarget`` 这类函数提到文件级（focus 就是这么干的）；
3. ****一个工具一个关注点****：宁可拆两个工具，不要把"查询"和"修改"塞进一个 ``action`` 枚举里（gov 的 9 个 action 已经是上限了，再复杂就该拆工具）。

****三个常见误区（对照检查）：****

1. ****"render" 就是给模型看的****——不完全对。`render` 决定的是界面/日志里的展示文本；模型看到的工具结果是规范值经展示层处理后的样子，但****结构化信息必须走 schema 字段****，别塞进 `render` 的字符串里让模型去解析；
2. ****"ctx.fs" 只是方便，我可以直接用 `node:fs`****——不行。宿主 `ctx.fs` 负责路径沙箱、containment、权限与审计，绕开它等于把工作区安全模型拆了；
3. ****"工具越全越好****——不是。工具是模型要读的 API 面，太多太杂会让模型选择困难。宁可少而精，把描述写透。

9. 进阶最佳实践清单

- [] `parameters` 用扁平 JSON Schema，动作收敛为 `enum`，字段写清"何时必填"
- [] 返回值是规范值（`output.schema` 约束），`render` 负责展示
- [] 用 `exec.agent.session.header.cwd` 定位工作区，不写死绝对路径
- [] 文件读写走 `ctx.fs`，且解析后过 `ctx.fs.contains` containment 校验
- [] 提示词段落用 `ctx.effect` 包裹，设置 `order`
- [] 事件钩子返回规范决策（`next() / {kind: "deny"}`），长任务响应 `signal`
- [] 纯逻辑（无 DSH/Cordis 依赖）抽到独立模块，便于单测
- [] 错误消息写人话，不抛 `undefined` 之类的裸异常

10. 下一步

工具写好了，怎么发布出去让全世界（以及 4300+ 插件生态）用？下一章

[06-发布插件](06-发布插件.md)：npm 发布、GitHub topic、awesome-dsh-plugin 精选列表 PR。

06 · 发布插件：npm、GitHub topic 与 awesome 精选列表

一句话总结：发布一个插件 = npm publish（token 是关键坑）+ 给仓库打 `dsh-plugin` topic + 向 awesome-dsh-plugin 提一个条目 PR（CI 会检查 bundle 声明、仓库年龄 ≥1 天、提交数 ≥10）；本章给出完整流程与检查清单。

1. 发布前检查清单

#	检查项	要求	位置
1	`dsh.bundle` 声明	必须有 `dsh.bundle.patch`（awesome CI 自动检查）	package.json
2	peerDependencies	官方 `@deepseek-ai/*` 一律 peer	package.json
3	`publishConfig.access`	"public"（scoped 包必须，否则 publish 报错）	package.json
4	`files` 白名单	只发 `lib`、`cordis.patch.yml`、README 等	package.json
5	README	有 README（含安装、用法、配置表）	仓库根
6	LICENSE	MIT 等	仓库根
7	测试	`node --check` 过、单测过（沙箱里用主模块方式跑）	—
8	包名	未被占用（`GET registry.npmjs.org/<name>` 返回 404 才可用）	—
9	仓库	已推 GitHub，≥1 天且 ≥10 commits（awesome CI 硬性要求）	GitHub
10	topic	仓库已打 `dsh-plugin` topic	GitHub

****为什么是这十项？**** 前六项保证"这个包是个合格的 npm 包"，第七项保证"代码能跑"，后三项保证"能被生态索引"（npm 不重名、GitHub 有门槛、topic 可被发现）。缺一项，要么发布失败，要么发出去没人能找到。****发布不是终点，能被找到才是****。建议把这张表存成仓库里的 `docs/RELEASE.md`（官方生态插件就是这么做的），每次发布前照着过一遍，比凭记忆可靠得多——漏掉 `publishConfig.access` 这种小字段，发布时才会在最后一步报错，返工成本反而最高。

2. npm pack --dry-run：发布前先看内容物

发布前先用 `pack` 预览 tarball 里到底有什么，防止把 node_modules、测试垃圾一起发出去：

```
npm pack --dry-run
npm pack --dry-run
```

****怎么读输出？**** 输出会列出将要打包的每个文件及大小。重点核对：

- 1. `cordis.patch.yml` 和 `lib/` 在不在？**少一个，装了就不生效**；
- 2. README / LICENSE 在不在？npm 页面上会展示；
- 3. 有没有不该出现的（`node\_modules/`、`.github/`、测试文件）？有就调整 `files` 白名单。

小技巧：`npm pack`（不带`--dry-run`）会生成一个`.tgz`，可以解压后逐文件检查，确认无误再删掉。这也是发布前最后一道“眼见为实”的关卡。另外在`pack`之前先确认一件事：****包名在 npm 上是否可用****。浏览器访问`https://registry.npmjs.org/<你的包名>`，返回404（JSON 报 not found）说明可用；返回 200 说明被占用，需要改名或加 scope（如`@你的名字/hello-plugin`）。这一步做晚了，`npm publish`会直接拒绝。

3. npm publish : token 与 2FA

3.1 token 类型 (403 重灾区)

npm 发布认证靠 token, 写在项目根目录的临时 `.npmrc` (记得 gitignore, 发布完删除) :

```
//registry.npmjs.org/:_authToken=<■■ token>
```

在 npm 官网 (npmjs.com → Access Tokens) 创建 **granular access token**, 关键选项:

选项	要求	否则报错
Packages	**All packages**（不能只选某个还不存在的包）	403 "may not perform that action"
Permissions	**Read and write**	403
Bypass 2FA for publishing	**必须勾选**	403 "2FA required"

三个 403 都是 token 配置问题，不是网络问题，重试无用——回去改 token 选项。****安全提示：**** token 不要粘贴到聊天/日志里，放 `$DSH_HOME/secrets/` 之类的本地文件，用完即弃；`.npmrc` 务必进 `.gitignore`。

****拿到一个正确 token 的完整步骤（一次配置，长期受益）：****

1. 打开 npmjs.com → 登录 → 右上角头像 → Access Tokens → Generate New Token ；
2. 类型选 ****Granular Access Token**** ；名字随意（如 ``dsh-publish``） ；
3. Packages 选 ****All packages**** （注意：如果你先发布了包再想收紧，之后可以再改；但发布前必须 All packages，否则 403） ；
4. Permissions 选 ****Read and write**** ；
5. 最关键的：在 ****Bypass 2FA**** 一栏勾选 ``Bypass 2FA for publishing`` ——这是给 CI/脚本发布用的授权，勾上后发布不再弹 2FA 验证 ；
6. 生成的 token 只显示一次，立刻复制保存到 ``$DSH_HOME/secrets/npm-token.txt`` （不要进 git、不要发聊天）。

之后每次发布：把 token 写进临时 `.npmrc` → `npm publish` → 删除 `.npmrc`。可以把这个流程固化成一个脚本（参考官方生态里 `scripts/publish-npm.ps1` 的做法），一次写好，以后一键发布。

3.2 执行发布

```
# ████ .npmrc██████████████████  
npm publish --ignore-scripts  
npm publish --ignore-scripts
```

- `--ignore-scripts`: 跳过生命周期脚本（如 `prepublishOnly` 里的测试）——沙箱环境常因 `npm` 子进程被拦截而失败，测试已经在本地手动跑过，这里跳过是安全的；

- 若 npm 缓存路径被沙箱拦截（EPERM），把缓存指到项目内目录：``npm publish --ignore-scripts --cache .npm-cache``；
- 发布成功后可立即验证：``npm view <你的包名> version``。发布失败的常见信号也要会读：403 系列看第 3.1 节的表；``ETARGET`` 之类表示版本号问题；网络超时则重试或换镜像。****发布是幂等友好的****：同名同版本会被拒绝（已存在），所以重发时记得先 ``npm version patch``。

3.3 版本管理

改代码后按 semver 递增版本再发布：

```
npm version patch
npm publish --ignore-scripts

npm version patch
npm publish --ignore-scripts
```

****版本语义建议****：0.x 阶段（0.1.0、0.2.0）随便改，``patch`` 修 bug、``minor`` 加功能；到了 1.0 之后，``major`` 才代表破坏性变更。DSH 官方包目前也是 0.1.0-rc.x 的 pre-release 节奏，跟随即可。

4. 给仓库打 dsh-plugin topic

``dsh-plugin`` topic 是插件市场的索引标签（可选附加：``deepseek-harness``、``cordis``）。有 GitHub CLI 的话一行搞定：

```
gh repo edit <owner>/<repo> --add-topic dsh-plugin
gh repo edit <owner>/<repo> --add-topic dsh-plugin
```

没有 gh：用 GitHub REST API ``PUT /repos/{owner}/{repo}/topics``（或 PATCH 的 ``topics`` 字段），认证用 classic ``repo`` token 或 fine-grained token（细节以 GitHub 文档为准）。

****为什么 topic 这么重要？**** 它是社区发现插件的第一入口（``github.com/topics/dsh-plugin``），也是 awesome 列表 CI 的检查项之一。打上 topic 后，你的仓库会立刻出现在 topic 页里——相当于给插件挂了“我是 DSH 插件”的招牌。

5. 提交 awesome-dsh-plugin PR

``awesome-dsh-plugin`` 是官方精选列表仓库，加条目走 PR。流程：

****① fork 并克隆****：

```
# ■ GitHub ■■ fork awesome-dsh-plugin/awesome-dsh-plugin ■■
git clone --depth 1 https://github.com/<■>/awesome-dsh-plugin.git
cd awesome-dsh-plugin

git clone --depth 1 https://github.com/<■>/awesome-dsh-plugin.git
cd awesome-dsh-plugin
```

网络不稳时 git 走 schannel 报 TLS 错误？加 ``-c http.sslBackend=openssl`` 再试，见 [08-常见问题](08-常见问题.md)。

****如果插件仓库还没推到 GitHub****，先补这一步：``git init -b main``

初始化、提交全部代码、创建远程仓库并推送。没有 gh CLI 时可以用 token 调 REST API 建仓，或用 GitHub 网页手动建空仓库后再 ``git remote add origin`` 推送。推送若遇 schannel TLS 问题，用 ``git -c http.sslBackend=openssl push`` 并配合一次性 token URL（推完把 remote 改回干净 URL），详见 08 章。

****② 添加条目****：新建 ``data/plugins/<owner>__<repo>.yaml``（****不要手改 README****）：

url: ``https://github.com/<owner>/<repo>``

```

name: <owner>/<repo>
category: memory # ui | theme | model | session | memory | tools | skill | workflow | notify |
description:
  en: 'One-line English description ending with a period.'
  zh: ████████████████████

```

****条目字段逐项说明：****

字段	说明
`url`	仓库地址，必须真实可访问
`name`	`<owner>/<repo>` 格式
`category`	十二选一：`ui` `theme` `model` `session` `memory` `tools` `skill` `workflow` `notify` `dev` `market` `fun`
`description.en`	一句话英文描述，**以句号结尾**（CI 可能检查标点）
`description.zh`	一句话中文描述，以句号结尾

****③生成 README：**** awesome 的 README 由脚本从 `data/plugins` 生成，**手改 README 会被 CI 打回**：

```

npm ci
node scripts/generate-readme.mjs --check

```

```

npm ci
node scripts/generate-readme.mjs --check

```

先跑 `node scripts/generate-readme.mjs` 重新生成 README，再跑 `--check` 确认同步。生成的 README 会同时出现在 `--check` 比对里，两处一致才算过。

****④提交并开 PR：****

```

git checkout -b add/<repo>
git add data/plugins/<owner>__<repo>.yaml README.md
git commit -m "add <owner>/<repo>"
git push origin add/<repo>

```

```

git checkout -b add/<repo>
git add data/plugins/<owner>__<repo>.yaml README.md
git commit -m "add <owner>/<repo>"
git push origin add/<repo>

```

然后在 GitHub 上对上游仓库开 PR：base `main`，head `<你>:add/<repo>`。分支命名用 `add/<repo>` 是惯例，方便维护者一眼看出"这个 PR 加了哪个插件"。

****为什么值得费这个劲提 awesome PR？**** 三个理由：其一，`awesome-dsh-plugin` 是官方精选列表，`data/plugins` 已有 1500+ 条目，被收录意味着你的插件进入了“官方认可的生态目录”，用户搜插件时能直接看到你；其二，列表的 README 会为每个条目生成一行描述，等于免费的曝光位；其三，**收录本身就是一次质量背书**——能过 CI 门槛（bundle 声明、≥1 天、≥10 commits、topic）的插件，至少不是三天热度的小玩具。当然，PR 被合并前你可能要等维护者 review，这是社区流程的正常部分。

6. CI 会检查什么（以及失败怎么救）

****一次完整的上架时间线（心里有个数）：**** 第 0 天：写完插件、本地验证全绿，推 GitHub（此时仓库年龄开始计时）；第 0~1 天：`npm publish` 发布 npm、打 topic；第 1 天以后（仓库满 24 小时、提交数 ≥10）：fork awesome 仓库、加条目、生成 README、开 PR；PR

合并后插件出现在精选列表。****唯一不能压缩的是时间****——age 检查必须等，所以别赶在发布当天就提 PR。

CI 检查	规则	失败处理
bundle 声明	仓库必须声明 `dsh.bundle`	补 package.json 后重推
仓库年龄	≥ **1 天**	等满 24 小时后， **在同一分支推空提交** 触发重跑：`git commit --allow-empty -m "ci: re-run"`，无需重开 PR
提交数	≥ **10 commits**	补提交（合入更多工作）后重推
topic	已打 `dsh-plugin`	打 topic 后重推
README 同步	与脚本生成一致	重跑 `generate-readme.mjs` 并提交

****为什么有年龄和提交数门槛？**** 这两个条件过滤"刚建仓就提 PR"的低质量条目，保证列表里的插件至少有真实开发历史。****age 检查失败不是你的代码问题，是时间问题****——等满 24 小时在同一分支推一个空提交触发 CI 重跑即可，规则允许达标后重新提交，没有惩罚，也不用重开 PR。

7. 发布后验证（闭环）

```
# ■ npm ■■■■■■■■■■■■
dsh plugin --profile web add <■■■■■>
dsh --profile web --dump-config | Select-String -Pattern "<■■■■■id>"

dsh plugin --profile web add <■■■■■>
dsh --profile web --dump-config | grep <■■■■■id>
```

装得回来、组合树里看得到、会话里调得动——一条插件就正式加入"一切皆插件"的生态了。****建议发布后 24 小时内在真实会话里完整跑一遍核心功能****：npm 的 tarball 与你本地目录可能有细微差别（`files` 白名单漏文件是最高频事故），只有从 npm 装回来测过才算闭环。具体验证路径：`dsh plugin --profile web add <包名>` 成功 → `--dump-config` 出现组合行 → 会话里让模型调用你的工具并核对返回值 → 再 `dsh plugin remove` 卸载确认清理干净。这套"装 → 查 → 用 → 卸"四步验证，以后每次发布新版本都值得跑一遍。

****本地安装与发布后安装有什么差别？**** 本地目录安装（`dsh plugin add <路径>`）时，DSH 直接引用你的源码目录——改代码重启即生效，适合开发。发布后从 npm 安装的是****打包后的 tarball****，只包含 `files` 白名单里的内容，且版本固定。所以：发布前务必 `npm pack --dry-run` 核对内容物，发布后务必从 npm 装回验证——****本地能跑 ≠ 发布后能跑****，`files` 漏文件是最高频的事故类型（比如忘了把 `cordis.patch.yml` 加进白名单，装回来插件直接不生效）。

8. 发布后的持续运营

发布不是终点。三条建议让你的插件长期"活着"：

- **维护 README 与版本节奏****：用户最先看的就是 README，安装命令、配置表、示例一个都不能少；每次功能变更记得 `npm version` 递增；
- **关注 issue 与 PR****：插件进入生态后会有用户提需求、报 bug，及时响应是口碑的来源；
- **迭代验证****：每次改完都走一遍"本地目录安装 → dump 验证 → 会话验证 → 发布 → 从 npm 装回验证"的闭环，防止 `files` 白名单漏文件之类的发布事故。

另外可以给仓库加上 `test.yml` / `publish.yml` 之类的 GitHub Actions（官方生态插件的标准配置）：CI 自动跑测试、打 tag 自动发布，人力成本降到最低。

9. 发布检查清单

- [] 十条发布前检查全过（含 `npm pack --dry-run` 内容物核对）
- [] 临时 `.npmrc` 用 granular token（All packages + Read and write + Bypass 2FA），用完删除
- [] `npm publish --ignore-scripts` 成功，`npm view` 可见
- [] 仓库打了 `dsh-plugin` topic
- [] awesome PR：条目文件正确、README 由脚本生成并 `--check` 通过
- [] 明白 CI 三项硬指标（bundle 声明 / ≥ 1 天 / ≥ 10 commits）与 age 失败的空提交重跑法
- [] 发布后从 npm 装回并验证生效

10. 下一步

纸上得来终觉浅。下一章 [07-实战案例](07-实战案例.md) 拆解三个真实插件（focus / recipe / gov）的完整设计，看高手怎么把本章和上一章的每个原则落进代码。

07 · 实战案例：拆解 3 个真实插件的设计

一句话总结：从 863683348 插件家族里挑三个代表——`dsh-plugin-focus`（专注板）、`dsh-recipe`（场景配方）、`dsh-gov`（治理套件）——看它们如何把前几章的原则落进代码：三件套结构、纯逻辑分层、schema 先行、ctx.fs 安全、生命周期管理与 fail-safe。

1. 三个插件一览

插件	一句话定位	核心工具	状态
`dsh-plugin-focus`	给 Agent 一块跨压缩、跨会话存活的"专注板"	`focus` (set/get/append/clear)	稳定 + 实验面板
`dsh-recipe`	"插件界的 dotfiles"：把插件打包成场景配方	`recipe` (list/search/apply/compose)	稳定
`dsh-gov`	企业配套：权限门禁 + 审计日志 + token 配额	`gov` (status/policy/audit/quota)	稳定

三者都已在 npm 发布、仓库打了 `dsh-plugin` topic、进入 awesome 生态，正好是第 06 章流程的活标本。下面逐个拆解：先讲它解决什么问题、怎么用，再讲代码层面怎么设计，最后提炼可复用的模式。

2. dsh-plugin-focus：持久化专注板

2.1 解决什么问题

Agent 干活时最怕两件事：**上下文被压缩**（compaction）丢掉关键约束，以及**换会话**忘记之前的目标。todo

列表管"下一步做什么"，专注板管"为什么做、什么不能漂移"——目标、硬约束、已做决定。focus 把这些信息持久化到工作区文件里，跨压缩、跨会话存活。

2.2 用法（模型侧）

`focus` 工具四个动作：

action	参数	行为
`set`	`note`（必填）	追加一条"当前焦点/约束"
`append`	`note`（必填）	追加日志条目（决定、发现、反转）
`get`	—	读整块板（最新在前、截断并注明省略数）
`clear`	—	清空；旧板归档到 `.dsh/focus.md.bak`（累积）

配合提示词段落 `focus:instructions`，模型被教会一套完整用法：任务开始或目标变化时 `set`；工作中 `append` 决定与发现；中断恢复或上下文压缩后 `get` 重读；任务结束 `clear`。

****一个使用场景：**** 你在一个大型重构任务里，模型刚说完方案，你告诉它“目标是把迁移做干净，不要动公共接口”。模型调用 `focus set` 把这条记进板子。之后上下文压缩了、甚至你换了个会话继续同一个 workspace，第一步模型就会 `get` 到板子，重新把约束捡回来——这就是“为什么做、什么不能漂移”被持久化的样子。

****一个使用场景：**** 你在一个大型重构任务里，模型刚说完方案，你告诉它"目标是把迁移做干净，不要动公共接口"。模型调用 `focus set` 把这条记进板子。之后上下文压缩了、甚至你换了个会话继续同一个

workspace，第一步模型就会`get`到板子，重新把约束捡回来——这就是"为什么做、什么不能漂移"被持久化的样子。

2.3 核心设计

****① 板子是纯文本文件。默认`.dsh/focus.md`，位于会话工作区；条目带时间戳按序排列，`get`最新在前、按`maxChars`（默认8000）截断视图但**永不删文件内容**。**文件格式示例：

```
# Focus Board

<!-- dsh-plugin-focus v1 -->

## [2026-08-14T23:12:00.000Z] set
<note■■■■■>

## [2026-08-14T23:13:00.000Z] append
<■■■■■>
```

****② 自动注入。复刻内置`dsh-time-context`机制：`agent/pre-step` waterfall 监听器在每轮开始（step 1）以及板子文本变化时，把板子作为 plugin snapshot 消息追加回模型上下文。会话恢复后第一步就能把板子带回来。板子是工作区级的：同一 workspace 的新会话能续上旧板子（每个会话从自己的`session.header.cwd`解析板子路径）。**

****③ `focusBoard` 会话投影。每次写入触发`focus/write`事件，UI 可用`useProjection('focusBoard')`读取；Web 面板（实验性）经包的`dsh.client` manifest 在同一组合行上加载`./client`——"两面一行"：宿主面加载`（工具+注入+投影）`，浏览器面加载`./client`（面板）。**

****④ 安全。所有文件访问走`ctx.fs`，每个路径解析后都做工作区 containment 校验（`ctx.fs.contains`），板子**永远逃不出工作区**；配置成工作区外路径会 fail loud 报错。**

****⑤ 纯逻辑分层。`lib/board.js`零 DSH/Cordis 依赖（解析/渲染/变更/归档，`parseBoard`/`renderBoard`/`applyAction`/`buildArchiveText`/`trimEntries`），独立单测；`lib/index.js`只做接线。**

2.4 配置表

键	默认	说明
`file`	`.dsh/focus.md`	板文件路径，相对会话工作区；禁止逃逸
`archiveFile`	`.dsh/focus.md.bak`	clear 的归档路径
`archive`	`true`	clear 是否先归档
`maxEntries`	`60`	磁盘最多保留条目数（最旧丢弃）
`maxChars`	`8000`	get / 自动注入的渲染上限（仅视图截断）
`autoInject`	`true`	每轮开始/板子变化时自动注入上下文
`personaSection`	`true`	注册提示词指引段落
`sectionOrder`	`5`	提示词段落顺序（persona 为 0，升序）

****值得学的点：板子按"工作区"而非"会话"存放；视图截断不删数据；clear 归档让"清空"可追溯；`autoInject`/`personaSection`做成可关闭的开关，给使用者留了降级路径。**

3. dsh-recipe：场景配方（插件界的 dotfiles）

3.1 解决什么问题

用户要的是**环境**（"浏览器开发环境"、"学术写作套装"、"通知全家桶"），不是一个一个插件。V2EX 有人做过"插件组合索引"但没做成可安装插件——recipe 把"装一套环境"做进 DSH 本体，一次对话拿到有序插件清单和安装命令。

3.2 用法（模型侧）

`recipe` 工具四个动作：

action	参数	行为
`list`	—	浏览全部配方
`search`	`need`	按自然语言需求匹配配方
`apply`	`recipeId` 或 `need`	展开配方：有序插件清单 + 安装命令序列
`compose`	`recipeId`（逗号分隔）	合并多个配方为整套环境（去重、保序）

内置 10 个配方：浏览器开发环境（4 插件）、通知全家桶（6）、学术写作套装（5）、移动远程控制（3）、安全加固套装（4）、记忆持久化套装（5）、娱乐休闲（4）、模型路由优化（5）、工作流自动化（5）、插件市场管理（5）。

3.3 核心设计

① 配方格式即数据。 `{ id, name, nameZh, description, descriptionZh, tags, plugins: [{ name, config? }]}`——依赖顺序就是数组顺序；`compose` 做并集去重，天然为 DAG 化编排留了前置。

② 纯逻辑 + 数据分离。 `lib/recipe.js`（检索/展开/组合/命令序列：`searchRecipes/expandRecipe/composeRecipes/installSequence`）零依赖；`lib/recipe-data.json`（配方）+ `lib/plugins.json`（84 个插件目录，与 dsh-need-finder 同源）。

③ 零副作用的安全声明。 无网络、无写入、无 secrets——它只"算"，不"动"。这带来两个好处：核心逻辑极好测（`node test/recipe.test.mjs`），模型可以放心调用（不怕它把环境搞坏）。

④ 输出即点菜单。 `apply/compose` 返回 `plugins`（有序清单）+ `commands`（逐条 `dsh plugin add` 命令）+ `oneLine`（一行合并命令），模型可直接执行或交给用户确认。

值得学的点："把环境当数据"——插件的价值可以只是**编排其他插件**；数据（`recipe-data.json`）与逻辑（`recipe.js`）分离，新增配方不用改代码。

4. dsh-gov：Agent 治理套件（企业级配套）

4.1 解决什么问题

企业引入 DSH 时第一波问题永远是三个：**谁能让 Agent 跑高危工具？出事了怎么追溯？钱花哪去了？** gov 把这三件事做成一个插件：策略门禁、结构化审计、token 配额，状态持久化在 `$DSH_HOME/gov/`（全局而非会话，跨工作区一致）。

4.2 三大支柱

支柱	机制	数据
策略门禁	按工具/agent/工作区规则，allow / deny / ask；`tool` 支持 `*` 通配；`priority` 高者胜，平局 fail-closed（deny > ask > allow）	`gov.json`

审计日志	每次工具调用（决定+结果）写入结构化 JSONL（`audit.jsonl`），可查询/导出/汇总	`audit.jsonl`
token 配额	接宿主 `tokenMeter`，按 agent 累计用量，周期配额（day/week/month/total），超限注入上下文警告，可一键重置	`gov.json`

4.3 接线方式（三个事件钩子）

```
// 1) tools/pre-execute———— deny/ask
ctx.on("tools/pre-execute", async (exec, next) => { /* decide + audit */ });

// 2) tools/result———— isError
ctx.on("tools/result", (exec, result) => { /* append audit */ });

// 3) agent/pre-step————ctx.tokenMeter.measure(...)
ctx.on("agent/pre-step", async (payload, next) => { /* quota */ });
```

`gov` 管理工具（9 个 action）：`status`（总览）、`policy\_list/add/remove`（规则）、`audit\_query/export`（审计）、`quota\_get/set/reset`（配额）。示例：

****一个治理会话示例**** 企业管理员先 `gov policy\_add tool="pwsh" policyAction=ask priority=10` 让所有 shell 命令需要审批，再 `gov quota\_set id=alice quotaLimit=100000 period=day` 给 alice 每天十万 token 预算。之后 alice 的 Agent 每次调用 pwsh 相关工具都会先被门禁拦下问询；一旦某天 token 超限，`agent/pre-step` 会在下一轮注入警告。管理者事后用 `gov audit\_query decision=deny` 就能看到所有被拒的调用——“谁、什么工具、为什么被拒”一目了然。

```
gov policy_add tool="pwsh" policyAction=ask reason="shell " priority=10
gov quota_set id=alice quotaLimit=100000 period=day
gov audit_query decision=deny limit=20
```

4.4 设计要点

- **fail-safe****：计量失败、持久化失败都**不阻塞 agent**（catch 兜底，继续放行）；策略引擎平局默认 fail-closed（deny > ask > allow）；
 - **审计不含参数内容****：只记工具名、agent、工作区、决定、理由、结果——刻意规避敏感信息落盘；
 - **纯逻辑分层****：`lib/policy.js`（规则引擎 `decide`）、`lib/audit.js`（审计模型）、`lib/quota.js`（配额计算）零依赖可单测；`lib/index.js` 只接线；
 - **提示词段落教模型****：被 deny 或配额警告**不要绕过**，去找用户调整（`GOV\_SECTION\_TEXT`）。
- **值得学的点**** 治理类插件 = 事件钩子 + 持久化 + 一个管理工具；"默认放行、规则收紧、平局 fail-closed"是平衡可用性与安全的成熟策略；审计日志是纯追加 JSONL，天然可被合规管道消费。

5. 三个插件共同的设计模式

模式	focus	recipe	gov
三件套结构（package.json + cordis.patch.yml + lib/index.js）			
纯逻辑分离（零 DSH/Cordis 依赖的 lib/*.js + 单测）	`board.js`	`recipe.js`	`policy/audit/quota.js`

动作收敛为 `enum` 的参数 schema	`action` 4 值	`action` 4 值	`action` 9 值
规范返回值 + `render`			
文件安全（`ctx.fs` + `containment`）	（工作区）	无文件（只算）	（`\$DSH_HOME/gov`）
生命周期管理（`ctx.effect` / `ctx.on`）			（3 个钩子）
fail-safe（失败不拖垮 Agent）		（零副作用）	（明确声明）
配置可开关（降级路径）	（autoInject 等）	少（sectionOrder）	（prestepWarn 等）

****给新手的三个取经结论：****

1. ****先分离纯逻辑****：把"解析/计算/渲染"写成不依赖 DSH 的函数，你就能在没有 DSH 的测试环境里把核心逻辑测到绿——三个插件全部如此，你的第一个插件也应该如此；
2. ****schema 先于实现****：把 `parameters`、`output.schema` 当接口契约先定死，实现只是填空；动作一律 `enum`，模型才调得准；
3. ****安全是默认项****：能碰文件就做 containment 校验，能碰钱（token）就做配额与审计，能做失败兜底就 fail-safe——这三个插件把"不出事"设计成了默认行为。

****怎么"抄作业"？**** 每个插件的仓库都是现成教材，建议按这个顺序读：`README.md`（定位与用法）→ `cordis.patch.yml`（组合行与默认配置）→ `lib/index.js`（接线）→ `lib/\*.js`（纯逻辑）→ `test/\*.test.mjs`（测试怎么写的）。看完照着写一遍，比任何教程都管用。

6. 本章小结

****三个插件三句话：**** focus 把"上下文工程"做成了持久化文件 + 自动注入；recipe 把"环境编排"做成了纯数据 + 纯逻辑；gov 把"企业治理"做成了钩子 + 持久化 + 管理工具。它们的共性不是代码多漂亮，而是****每一条都踩在前几章的原则上****：三件套结构让它们可安装，纯逻辑分层让它们可测试，schema 先行让模型调得准，containment 与 fail-safe 让它们出不了大事。

****给你的一句话行动建议：**** 读完本章后，去 GitHub 打开这三个仓库之一（推荐从 `dsh-plugin-focus` 开始），按 05 节建议的顺序读一遍源码，然后动手实现一个 20 行以内的小工具插件——等你亲手跑通 `--dump-config` 看到自己插件的组合行那一刻，DSH 插件开发对你就不再是"教程里的概念"，而是"手上的工具"了。

`focus` 展示了"持久化 + 注入"的上下文工程，`recipe` 展示了"插件编排插件"的数据化思维，`gov` 展示了"钩子 + 持久化 + 管理工具"的治理结构。三者共同验证了本书前六章的每一条原则都可以落地。下一章 [08-常见问题](08-常见问题.md) 是踩坑速查表——Windows 路径、沙箱 EPERM、npm 403、git TLS……写插件路上会遇到的坑都在那里。

08 · 常见问题：踩坑速查表

一句话总结：本章把 DSH 开发中最常踩的坑按"症状 → 原因 → 处理"整理成速查表——Windows 路径引号、沙箱 EPERM、npm 403、git TLS、awesome CI 年龄检查、PowerShell 转义、ctx.fs 权限——遇到先查表，别自己瞎试。

1. 踩坑速查总表

症状	原因	一句话处理
PowerShell 里路径带 ` ` (如用户目录 `%userprofile%`) 报错	单引号字符串被路径里的引号截断	单引号写成 ` ` (两个单引号) 或改用双引号
`node --test` 报 EPERM / 子进程被拦截	沙箱拦子进程管道	改主模块方式跑测试文件：`node test\xxx.test.mjs`
npm 403 `2FA required`	用了 classic token + 2FA	granular token 并勾选 `Bypass 2FA for publishing`
npm 403 `may not perform that action`	granular token 只选了 select packages	换成 `All packages` 的 token
npm EPERM 缓存路径	沙箱拦 `%appdata%\npm-cache`	加 `--cache <项目>\.npm-cache`
git 克隆/推送报 TLS/schannel 错误	网络环境证书问题	`-c http.sslBackend=openssl`
awesome CI 的 age 检查失败	仓库创建不满 1 天	满 24h 后在原分支推空提交重跑
插件路径"逃逸工作区"报错	配置路径在工作区外	改为相对工作区的路径；文件读写走 `ctx.fs`
PowerShell 命令里 `\$`、引号行为怪异	变量展开与转义	用反引号转义 (PowerShell) 或单引号字符串
`--dump-config` 输出中文乱码	PowerShell 重定向默认编码	用 `Out-File -Encoding utf8`
模型"看不到"新装的工具	没重启，或 description 太含糊	重启 DSH；把工具描述写具体

下面逐项展开。

2. Windows 路径里的单引号 (` ` 之类)

症状： 路径含单引号时 PowerShell 报"字符串缺少终止符"或路径解析错误。典型例子：用户目录 `%userprofile%`。

原因： PowerShell 单引号字符串里再出现单引号会提前终止字符串。

处理： 把路径里的单引号写成`两个单引号` (` ` 是 PowerShell 转义单引号的写法)，或整个字符串改用双引号：

```
# 1 l'x
cd 'C:\Users\l'x\WorkBuddy\demo'

# 2 1
cd 'C:\Users\l' 'x\WorkBuddy\demo'

# 3 2 $ $
cd "C:\Users\l'x\WorkBuddy\demo"

# bash
cd "/c/Users/l'x/WorkBuddy/demo"
cd '/c/Users/l' "'x/WorkBuddy/demo'
```

在 Node/JS 代码里遇到这种路径则没有这个问题，字符串字面量照常写。****预防建议：****
给项目建一个不含单引号的短路径（如 `D:\dev\<项目>`），从源头避免这类转义地狱。

****同类问题：**PowerShell 重定向中文乱码。****`dsh --profile web --dump-config > dump.yml`** 在 Windows PowerShell (5.x) 里默认用 UTF-16 写文件，内容打开变乱码，bash 则没这个问题。统一用 `Out-File -Encoding utf8` 解决（08 章的转义对照表里也列了这条）。

3. 沙箱 EPERM：node --test 的子进程被拦截

****症状：**** 在受限环境（CI 沙箱、DSH 自身的文件沙箱）跑 `node --test test/` 报 EPERM，或提示子进程管道被拒绝。

****原因：**** 沙箱禁止通过管道捕获子进程输出（`node --test` 会 spawn 子进程），这是设计好的边界，不是 bug——****不要反复重试同一命令****，改用法。

****处理：**** 改用****主模块方式****直接跑测试文件（`node:test` 文件本身可以当主模块执行）：

```
# ■■■■■node --test test/
node --check lib\index.js
node test\board.test.mjs
node test\recipe.test.mjs

node --check lib/index.js
node test/board.test.mjs
node test/recipe.test.mjs
```

发布时同理：`npm publish --ignore-scripts`（测试已在本地手动验证），避免 npm 生命周期脚本在沙箱里触发子进程。

****相关症状：**模型“看不到”新装的工具。******装了插件、dump 里也有，但模型就是不用。先查两件事：第一，****重启了没有****——配置树在启动时组合，运行中的会话不会加载新插件；第二，****工具描述（description）不够具体****——“提供 focus 功能”这种描述模型不知道怎么触发，改成“当需要记录任务目标/约束/决定时调用”这类“何时用”的表述。05 章讲过：description 是模型的说明书。

4. npm 403

npm 发布 403 基本都不是网络问题，重试无效。按错误信息对号入座：

错误信息	原因	处理
`403 2FA required`	token 没被授予跳过 2FA 的权限	建 token 时勾选 **Bypass 2FA for publishing**
`403 may not perform that action`	granular token 只授权了 select packages，而目标包不存在/不在清单	换成 **All packages** 范围
`403 You must be logged in`	token 没写进 `.npmrc` 或格式不对	确认 `//registry.npmjs.org/:_authToken=<token>`

****推荐配置：**** npm 官网创建 ****granular access token****：Packages = All packages，Permissions = Read and write，勾选 Bypass 2FA for publishing。token 放本地文件（如 `\$DSH\_HOME/secrets/npm-token.txt`），****不要粘贴到聊天记录或提交进 git****。

****验证是否解决：**** 改完 token 后重新执行 `npm publish`，若仍 403 则去 npmjs.com 的 Access Tokens 页面核对勾选选项——最常见的坑是忘了勾 Bypass 2FA。

5. npm EPERM 缓存路径

****症状：**** 发布或安装时报 `EPERM ... npm-cache`。

****原因：**** 沙箱拦截了默认缓存目录（Windows 上 `AppData\npm-cache`）。

****处理：**** 把缓存指到项目内目录：

```
npm publish --ignore-scripts --cache .npm-cache
```

```
npm publish --ignore-scripts --cache .npm-cache
```

（`.npm-cache` 记得加进 `.gitignore`。）这条对 install/pack/publish 都适用，沙箱里跑 npm 相关命令时养成加 `--cache` 的习惯。

6. git schannel TLS 失败

****症状：**** `git clone` / `push` 报证书/TLS 错误（Windows 上常见：curl schannel 校验失败）。

****原因：**** 网络环境/系统证书问题导致 Windows 默认的 schannel 后端握手失败。

****处理：**** 强制改用 OpenSSL 后端：

```
git -c http.sslBackend=openssl clone https://github.com/<owner>/<repo>.git
```

```
git -c http.sslBackend=openssl push origin <branch>
```

```
git -c http.sslBackend=openssl clone https://github.com/<owner>/<repo>.git
```

极少数情况下还需要配合一次性 token URL 推送（推完把 remote 改回干净 URL）。****预防建议：**** 一次性命令成功后，可以 `git config --global http.sslBackend openssl` 固定后端，避免每条命令都带参数。

7. awesome-dsh-plugin CI 的 age 检查失败

****症状：**** PR 的 CI 报 "repository must be at least 1 day old"（或 commits 不足）。

****原因：**** awesome 列表的三条硬性规则：必须声明 `dsh.bundle`（自动检查）、仓库 $\geq$ ****1 天**** 且 $\geq$ ****10 commits****、已打 `dsh-plugin` topic。刚建仓立刻提 PR 必挂。

****处理：****

- 提交数不足 → 把真实工作合入仓库（ ≥ 10 commits）后重推；
- 年龄不足 → ****等满 24 小时****，然后****在同一分支推空提交****触发 CI 重跑，****无需重开 PR****：

```
git commit --allow-empty -m "ci: re-run"
```

```
git push
```

```
git commit --allow-empty -m "ci: re-run"
```

```
git push
```

规则允许达标后重新提交，没有惩罚。****心态提示：**** age

失败是时间问题不是质量问题，别为它熬夜，设个 24 小时的提醒即可。

8. PowerShell 路径与命令转义

PowerShell 与 bash 的转义规则不同，常见差异：

场景	PowerShell	bash
字符串里嵌双引号	外层用单引号	转义 `\"` 或外层单引号
字符串里嵌单引号	写两个 `''`	双引号包住
变量	`\$env:NAME`（环境变量）	`\$NAME`

特殊字符（如 `\$`）	用反引号转义	用反斜杠转义
输出重定向到文件	`Out-File -Encoding utf8`（避免中文乱码）	`>`

例如保存 `--dump-config` 输出：

```
dsh --profile web --dump-config | Out-File -FilePath dump.yml -Encoding utf8
dsh --profile web --dump-config > dump.yml
```

****一句话记法****：PowerShell 的转义字符是****反引号****，bash 的转义字符是****反斜杠****；PowerShell 里变量用 `\$env:NAME` 读环境变量。搞混了就去对照表，别硬背。

****路径书写的一个统一约定（少踩一半坑）****：在给读者/文档写命令时，Windows 命令里用反斜杠（`lib\index.js`），bash 里用正斜杠（`lib/index.js`）；但在 ****Node/JS 代码里****（插件源码、测试），一律用正斜杠或 `node:path` 的 `join`——因为 JS 字符串里的反斜杠是转义符，`'lib\index.js'` 会变成 `libindex.js` 之类的鬼东西。这条规则记住，Windows 相关转义问题直接少一半。

****PowerShell 执行策略（偶尔碰到）****：跑 `ps1` 脚本（比如发布脚本）报“禁止运行脚本”，是执行策略限制。查看 `Get-ExecutionPolicy`，临时放行用 `Set-ExecutionPolicy -Scope Process Bypass`（只影响当前窗口）。这是 PowerShell 通用知识，不是 DSH 特有，但写插件发布脚本时会遇到。

****WSL 用户的提示****：WSL 里 dsh 与 npm 都按 Linux 方式工作（`~/dsh`、正斜杠路径），Windows 侧的坑基本不沾；但注意 WSL 与 Windows 的文件系统互访（`/mnt/c/...`）会让路径很长，建议在 WSL 自己的文件系统里建项目。

****常见启动问题速答****：

- ****端口被占用****：`dsh web` 起不来，终端报 EADDRINUSE → 换端口：`dsh --profile web --port 8080`；
- ****profile 不存在****：`dsh --profile mydev` 报错 → 自定义 profile 必须先用 `dsh plugin --profile mydev ...` 创建（web/headless 才自动初始化）；
- ****改了配置没生效****：先 `--dump-config` 确认值，再确认改对了文件（profile 级 vs home 级），最后确认重启了。

9. ctx.fs 权限与 containment

****症状****：插件报 `escapes the session workspace`，或读写被拒。

****原因与规则****：

- ****插件文件读写必须走宿主 `ctx.fs`****（`resolve` / `stat` / `readText` / `writeText` / `contains`），不要直接 `node:fs`；
- 配置路径解析后必须过 `ctx.fs.contains` 工作区校验——配置成工作区外路径会 fail loud（刻意设计，见 05 章）；
- `exec.agent.session.header.cwd` 是会话工作区，路径以它为基准；
- 全局数据（如 dsh-gov 的 `\$DSH\_HOME/gov`）走 `dshHomePath` 之类的宿主能力，不要硬编码拼接。

****排查顺序：****① 是不是直接用了 ``node:fs`` → 改走 ``ctx.fs``；② 路径是否相对工作区 → 改成相对路径；③ 是否越权写 ``$DSH_HOME`` 之外 → 用宿主提供的 home 路径能力。仍不确定的以官方文档为准。

10. 调试三板斧

****三板斧的顺序是有道理的：****先确认"配置层没问题"（1），再确认"代码层没问题"（2），最后才是"运行时行为"（3）——按这个顺序排查，80% 的插件问题能在前三分钟定位。

****一个完整的排查示例（照着练）：****你刚写完插件，``dsh plugin add`` 成功，但会话里模型说没有你的工具。按三板斧走：① ``--dump-config | Select-String <你的id>``——没有输出？先查 ``cordis.patch.yml`` 的 ``insert`` 缩进和 ``id`/`name`` 一致性；② 有输出但模型还是看不到？``node --check lib\index.js`` 过不过、``inject`` 里声明的服务有没有写全、``apply`` 有没有真的调用 ``ctx.tools.register``；③ 都正常？重启后直接在会话里命令模型"调用 xxx 工具"，看它报什么错——工具调用失败的错误信息会直接暴露问题点。

****怎么向别人求助（带上这三样）：****① 你的 ``--dump-config`` 输出（截取相关片段）；② ``node --check`` 结果与插件目录结构；③ 会话里模型调用工具时的报错原文。这三样齐了，任何熟悉 DSH 的人都能快速定位——比一句"我的插件不生效"有效一百倍。

遇到任何"插件没生效/行为怪异"，按顺序做：

1. ****看组合树****：``dsh --profile <name> --dump-config``——插件行在不在？config 对不对？
2. ****看语法****：``node --check lib\index.js``——语法错会在这里现形；
3. ****看会话行为****：重启后在会话里让模型调用该工具，观察返回的规范值与错误信息。

****三板斧的顺序是有道理的：****先确认"配置层没问题"（1），再确认"代码层没问题"（2），最后才是"运行时行为"（3）——按这个顺序排查，80% 的插件问题能在前三分钟定位。

****速查表的使用姿势：****别等出事了才翻——把第一章的“新手检查清单”和本章的总表各打印/收藏一份，每次装插件、写插件、发插件之前先过一眼。坑是踩不完的，但****高频的坑就这几类****，覆盖了它们，你的 DSH 之旅会顺畅得多。

11. 写在最后

本书八章走完了"跑起来 → 理解 → 找/装 → 写 → 进阶 → 发布 → 案例 → 排障"的完整路径。DSH 生态还在高速增长（72 小时 4300+ 插件、awesome 列表 1500+

条目），官方迭代也很快——****本文所有命令与 API 以你安装的 dsh 版本与官方文档为准****。祝你在"一切皆插件"的世界里玩得开心。如果读到这里你还没有亲手装过一个插件，现在就去执行第一章第 8 节的示例——十分钟后回来看，收获会完全不同。